2026 年全国大学生物理实验竞赛

（创新）

命题类创新作品

题目六：AI+物理实验

智测微光：AI 辅助迈克尔逊白光干涉实验

研究报告

2026 年 9月

摘　要

针对传统迈克尔逊干涉仪白光等厚干涉实验中等光程位置难以寻找、中心条纹判读依赖经验、数显读数易产生抄录误差以及教学过程缺少实时指导等问题，本作品设计并研制了一套基于人工智能与计算机视觉的迈克尔逊白光干涉智能辅助实验系统。系统由三部分构成：（1）基于大语言模型与 RAG 本地知识库的流程型实验助手，结合只读设备状态提供预习、操作、误差分析和报告整理建议；（2）基于 YOLO 与工业相机的视觉系统，完成条纹候选区域识别、中心位置精修以及数显微分表 OCR 读数；（3）基于 C51 单片机、步进电机和数显微分表的动镜辅助调节模块，在人工选择初始搜索方向后寻找稳定条纹，并在识别到中心条纹后切换为闭环寻中。

系统将双相机画面、条纹识别、读数记录、电机状态和实验指导集中在同一界面中。条纹侧通过连续帧跟踪抑制单帧抖动，读数侧通过 LCD 定位、数字识别和多帧稳定输出过滤突变；控制侧设置人工停止、运行超时、串口异常和目标丢失保护。该设计在保留学生光路预调和方向判断的基础上，降低了重复观察与机械调节负担。装置采用可拆卸改造，硬件成本约 1200 元，适用于高校物理实验教学与竞赛展示。

关键词：迈克尔逊干涉仪；白光干涉；AI 智能体；YOLO 目标检测；计算机视觉；实验教学辅助

目录

摘　要 2
目录 3
第一章　　引言 5
1.1 题意分析 5
1.2 研究背景与意义 5
1.3 现有实验存在的问题 6
1.4 本作品的研究思路 6
第二章　　研究方案 7
2.1 实验选择——迈克尔逊干涉仪 7
2.2 白光干涉的特点 7
2.3 传统实验痛点分析 8
2.4 研究方案 9
2.5 实验装置 10
第三章　　实验原理 12
3.1 白光等厚干涉 12
3.1.1 白光等厚干涉原理 12
3.1.2 白光光源特性与选型 13
3.1.3 楔角与等厚条纹宽度的定量关系 15
3.1.4 已知折射率的介质片厚度测量 15
3.1.5 介质片旋转角度测量 16
3.2 AI 应用原理 17
3.2.1 YOLOv8 目标检测原理 17
3.2.2 OCR 读数原理 18
第四章　　基于 AI 的辅助系统设计 19
4.1 智能体开发 19
4.2 视觉方案设计 20
4.2.1 干涉条纹识别 20
4.2.2 中心条纹识别 23
4.2.3 微分表视觉读数 25
4.3 动镜辅助调节系统设计 26
第五章　　测量精度与不确定度 28
5.1 实验误差分析 28
5.2 条纹定位精度的不确定度测量 29
5.2.1 测量模型 29
5.2.2 测量方法与数据 30
5.2.3 标准不确定度的 A 类评定 30
5.2.4 标准不确定度的 B 类评定 30
5.2.5 合成标准不确定度 31
5.3 介质厚度测量与不确定度分析 32
5.3.1 测量模型 32
5.3.2 测量方法与数据 32

5.3.3 标准不确定度的 A 类评定 33
5.3.4 标准不确定度的 B 类评定 33
5.3.5 合成标准不确定度 33
5.4 微小转角测量与不确定度分析 34
5.4.1 测量模型 34
5.4.2 标准不确定度的 A 类评定 34
5.4.3 标准不确定度的 B 类评定 35
5.4.4 合成标准不确定度 36
5.5 彩色白光干涉的盖玻片厚度起伏辅助分析 36
5.5.1 理论分析 36
5.5.2 最终项目中的程序功能 37
5.5.3 示例结果 37
5.5.4 不确定度传播 38
第六章 实验创新点与总结 40
6.1 实验创新点 40
6.2 总结 40
第七章 参考文献 42
附录一：成本清单 44
附录二：成员分工 45
附录三：实验选用工业相机完整参数表 46
附录四 实验相关代码 47
附录 4.1 YOLO 模型加载与条纹目标检测 47
附录 4.2 中央条纹定位与连续帧跟踪 49
附录 4.3 运动过程条纹增强 53
附录 4.4 数显微分表定位、OCR 识别与稳定读数 54
附录 4.5 已知方向搜索与条纹闭环寻中 59
附录 4.6 电机串口连接与运动控制 62
附录 4.7 DeepSeek 实验助手问答 65
附录 4.8 条纹仿真 67

第一章　引言

1.1 题意分析

题目六：AI+物理实验

目的：

将人工智能技术与物理实验深度融合，实现物理现象观察、关键参数测量、实验过程辅助与结果分析。

要求：

(1)设计实验方案（含实验原理）；

(2)制作实验装置，实现物理现象观察或关键物理参数测量；

(3)利用 AI 技术辅助测量过程、数据处理或结果分析；

(4)结合实验结果讨论测量精度与不确定度。

作品的设计目标从三个维度展开：第一，立足迈克尔逊白光等厚干涉原理，准确观察干涉现象并表征关键位置；第二，使人工智能与视觉检测、运动控制和实验指导有机协同，避免功能简单堆砌；第三，面向高校物理实验教学，兼顾可操作性、可解释性与安全性。

1.2 研究背景与意义

迈克尔逊干涉仪是大学物理实验中的经典光学仪器。通过观察干涉条纹的形态及其变化，可以理解光的相干性、光程差与干涉现象之间的关系，并可用于微小位移、波长和透明介质参数等物理量的测量。该实验兼具理论学习和实验能力训练价值，在大学物理实验教学中具有重要地位。

白光等厚干涉具有中央条纹位置明确、对光程差变化敏感等特点，可用于确定等光程位置及开展微小位移测量。然而，白光的相干长度较短，只有当干涉仪两臂的光程非常接近时，才能观察到清晰的彩色条纹。实验过程中需要进行较为精细的光路调

节和动镜控制，对实验者的操作经验和观察能力要求较高。

将人工智能技术引入这一实验，有助于提高条纹观察和位置判断的一致性，减少重复调节与人工读数带来的负担。同时，人工智能还可以根据实验状态提供过程指导，使学生更加清楚地理解当前实验阶段、下一步操作及相应的物理现象。

1.3 现有实验存在的问题

传统迈克尔逊白光干涉实验主要依赖人工观察和手动调节。由于白光干涉条纹出现范围较窄，学生在寻找等光程位置时容易出现调节过快、错过目标位置或无法准确判断中央条纹等问题。机械回程、环境振动和光照变化也会影响条纹的稳定性，使不同实验者得到的结果存在差异。

实验过程中，学生还需要同时关注干涉画面、微分表读数和操作步骤，容易发生读数错误或记录遗漏。教师难以在有限时间内持续掌握每组实验的具体状态，也难以针对不同学生提供实时指导。因此，有必要设计一套能够辅助观察、调节、读数和实验指导的智能系统。

1.4 本作品的研究思路

针对上述问题，本作品设计了一套迈克尔逊白光干涉智能辅助系统。系统利用相机采集干涉条纹和数显微分表画面，通过计算机视觉识别中央条纹并读取仪器数据，同时结合电机控制辅助调节动镜位置。

在实验教学方面，系统设置实验助手，根据实验原理、操作步骤和当前实验进度提供针对性指导。实验者仍需完成光路预调、光源切换和实验现象判断，人工智能主要承担条纹识别、状态分析和信息整理等辅助任务。

本作品旨在降低白光干涉实验的操作门槛，提高实验过程的可观察性和测量结果的一致性，同时保留学生对实验原理、光路变化和干涉现象的主动分析过程，为人工智能技术在大学物理实验教学中的应用提供参考。

第二章　研究方案

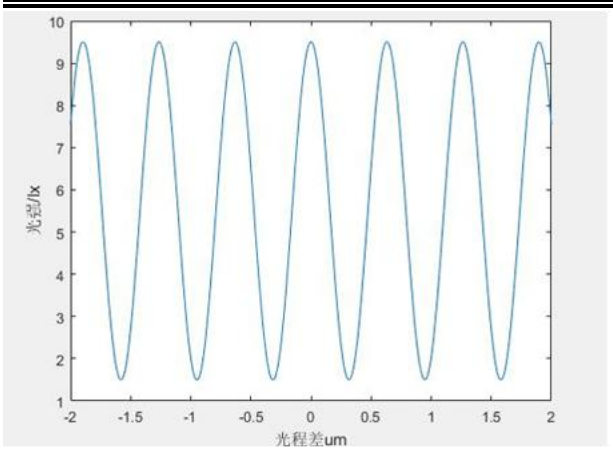
2.1 实验选择——迈克尔逊干涉仪

迈克尔逊干涉仪是美国物理学家迈克耳孙于 1883 年设计的分振幅双光束干涉器，是经典的大学物理实验，包括点光源非定域干涉、白光等厚干涉等内容。其应用十分广泛，主要涉及长度的精密测量与校准、光谱学与波长分析、引力波探测等领域。

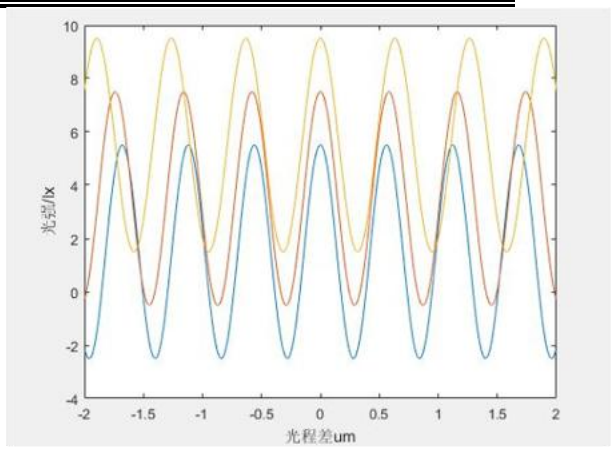
在白光等厚干涉实验中，存在学生上手难度普遍较大等诸多问题，实际教学中常作为拓展内容出现，亟待系统性地优化与革新。因此，本作品选择迈克尔逊干涉仪白光干涉实验作为 AI 技术赋能的对象。

2.2 白光干涉的特点

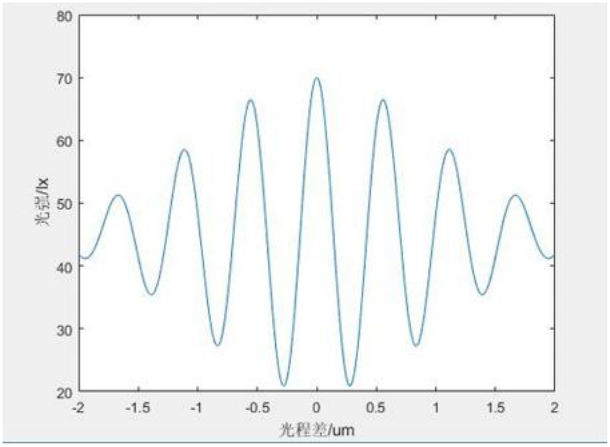
白光干涉具有双重特性。一方面，白光干涉具有高分辨率、强抗干扰能力、精度高的优点，在微小位移测量等领域应用广泛。另一方面，白光是复色光，相干长度在微米量级，只有当动镜 M1 和虚镜 M2’之间的距离 d 在等光程位置——即动镜 M1 和虚镜 M2’的相交线附近几个微米之内时，才能观察到白光干涉彩色条纹。在 d=0 附近，可用白光光源观察到干涉条纹，如图 2.1.1[1]所示，这些条纹是由不同单色光干涉结果叠加而成的，除中央条纹外，其他条纹都是左右对称的彩色条纹。



单色光干涉光强



不同单色光干涉光强



各种波长的单色光干涉光强叠加

图 2.2.1 白光各波长干涉光强叠加及相干长度示意图

相干长度小、等光程点外光强衰弱快的特性，使实验中观察到彩色干涉条纹难度大，对实验人员的调节精度、专注能力和操作手法有很高的要求。为此引入 AI 辅助，以达成快速准确获取干涉条纹的目的。

2.3 传统实验痛点分析

通过对实际教学过程的梳理，归纳出三大核心痛点：

问题一：教学答疑资源紧缺。

学生在实验预习和操作过程中存在诸多疑问，如动镜的速度与移动方向不明确、对核心原理理解不充分（如中心条纹为什么是黑色），但缺少可靠的答疑途径和资源。同时，对激光预调不理解、不知道何时应该换成白光源等问题也普遍存在，教师实际

指导压力较大。

问题二：实验过程重复、反馈不足。

实验操作中，学生长时间观察不到条纹易失去耐心；手调千分尺误差较大；手调数据得不到检验校准，误差较大；实验报告撰写困难，数据复杂度高、计算困难。在拓展实验（如测量薄膜厚度）中，手调误差较大且测量时间久，多次测量耗费大量时间。

问题三：装置数字化与自动辅助程度较低。

现有实验装置缺乏自动化辅助手段，从实验预习、实验复习到实验操作整个过程依赖人工指导和手动操作，效率和一致性有待提升。

2.4 研究方案

针对上述问题，本作品采用“教学指导—视觉感知—运动执行”三层协同方案。

- 开发学生端智能体辅助实验，缓解高校教师指导资源紧缺。通过大语言模型加本地知识库（RAG）构建流程型智能体，贯穿实验全部流程，提供预习指导、过程辅助、误差计算和报告生成功能。



图 2.4.1 学生端智能体图片

- 训练 YOLO AI 中心条纹识别模型，辅助寻找条纹中心。基于工业相机采集干涉条纹图像，通过 YOLO 神经网络实时识别条纹类别与状态。

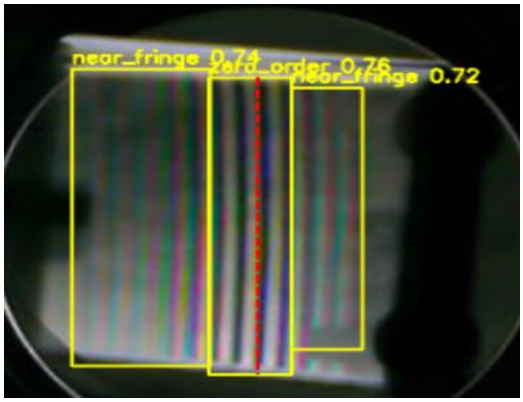


图 2.4.2 YOLO 模型辅助寻找条纹中心

- 电机驱动千分尺调节，视觉系统实时回传位置，实现图像与位置的实时统一。融合 C51 单片机、步进电机和数显千分尺，使动镜调节实现电动化和数字化。





图 2.4.3 装置改造前后对比

2.5 实验装置

实验装置（俯视）由以下部分组成：①白炽灯光源、②迈克尔逊干涉仪、③CCD 相机、④动镜辅助调节模块、⑤升降台、⑥计算机处理终端。其中，CCD 相机用于采集干涉条纹图像和数显千分尺读数，动镜辅助调节模块包含步进电机、C51 单片机和数显千分尺，计算机处理终端运行 AI 智能体和视觉识别程序。

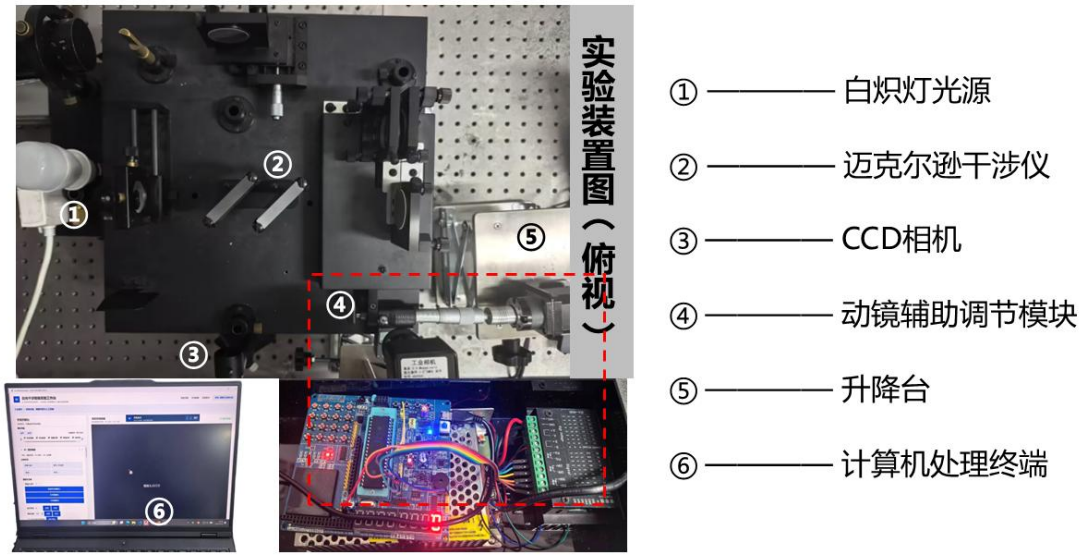


图 2.5.1 实验装置俯视图及主要组成

第三章　实验原理

3.1 白光等厚干涉

3.1.1 白光等厚干涉原理

当 M_1 与 M_2'之间有一个很小的角度α时，M_1与 M_2'间为一楔形空气薄层，如图 3.1.1.1 所示。设某光线经两个面反射，得光线(1)和(2)。它们在镜面附近相交，产生定域干涉。当α很小时，如前所述，由一条入射光线反射出的光线(1)和(2)的光程差，可近似地表示为 $\delta = 2dcos\theta$，其中 d 是观察点附近的空气层厚度，θ 为入射角。由于两个反射面都是平面，d 相同的点在同一条直线上，看到的应是直条纹。

在 M_1 与 M_2'相交 M 处，d=0，再考虑到分束镜表面反射时的半波损失，应为暗条纹，称中央条纹。当 θ 角较小时，有

$$cos\theta \approx 1-\frac{1}{2}\theta^2, \delta \approx 2d\left(1-\frac{1}{2}\theta^2\right) = 2d - d\theta^2 \quad (1)$$

在迈克尔逊干涉仪中，G_1为分光板，G_2为补偿板，M_1为定平面镜，M_2为动平面镜，M_2'为M_2动镜所成虚像。设α为平面镜 M_1与M_2'之间的夹角，d 为 M_1与M_2'之间空气层厚度。白光光源经毛玻璃片后成为扩展光源，照射在迈克尔逊干涉仪上，经 M_1和M_2反射后形成两束相干光，在 CCD 相机所在平面产生干涉，详细光路可参考图 3.1.1.2[2]。

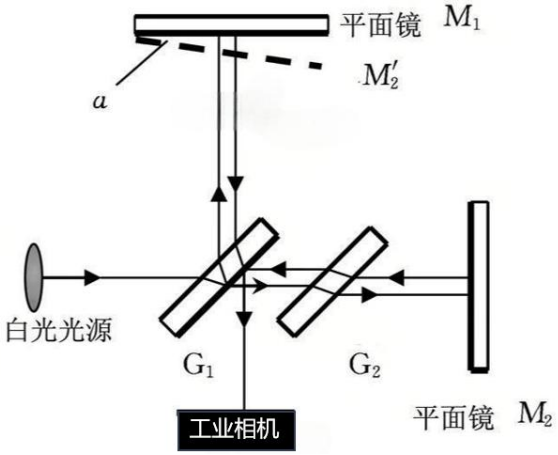


图 3.1.1.1 迈克尔逊白光干涉实验光路图

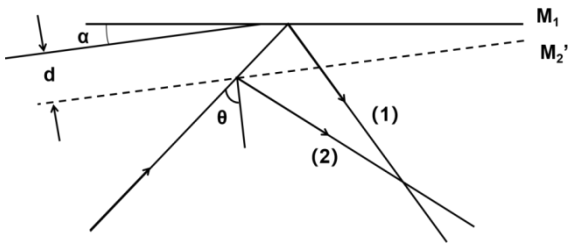


图 3.1.1.2 等厚干涉示意图

在 d=0 附近（即等光程位置附近），可用白光光源观察到干涉条纹。这些条纹是由不同波长的光干涉结果叠加而成的。除中央条纹外，其他条纹都是左右对称的彩色条纹。由于白光的相干长度在微米量级，条纹只在两镜面相交线附近极小的区域内可见，因此寻找白光干涉条纹本质上是在寻找等光程位置。

3.1.2 白光光源特性与选型

白光等厚干涉的可见范围、颜色分布和中心条纹辨识度与光源的光谱结构密切相关。为选择适合视觉识别和自动寻中的光源，对卤素灯、白炽灯和 LED 灯的光谱及实际条纹表现进行了对比[3]。

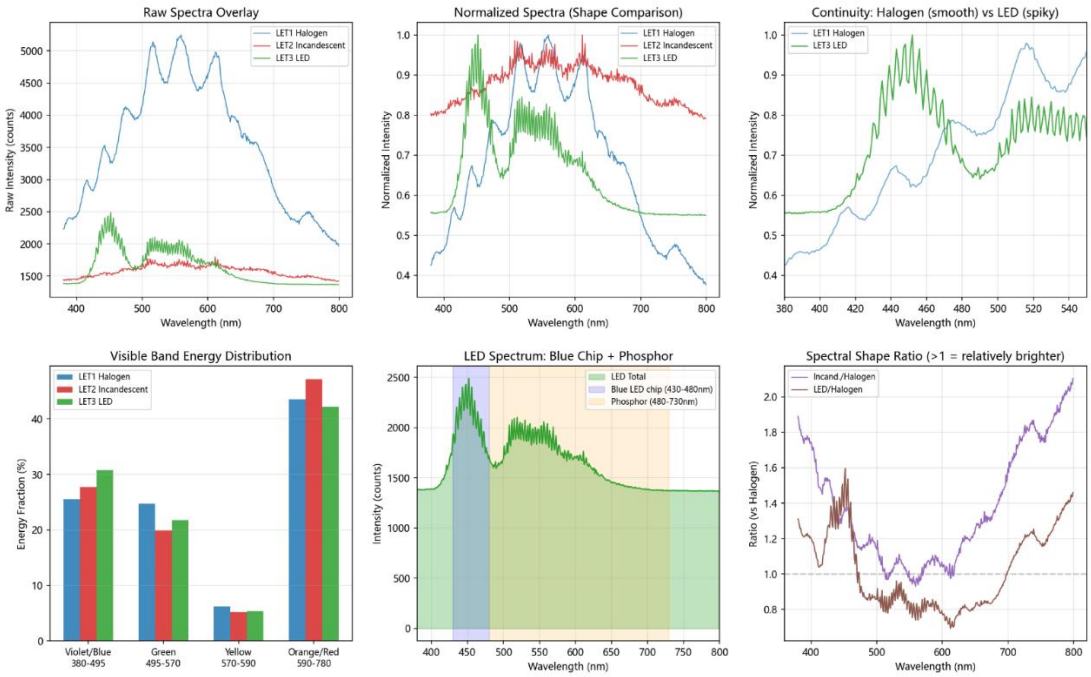
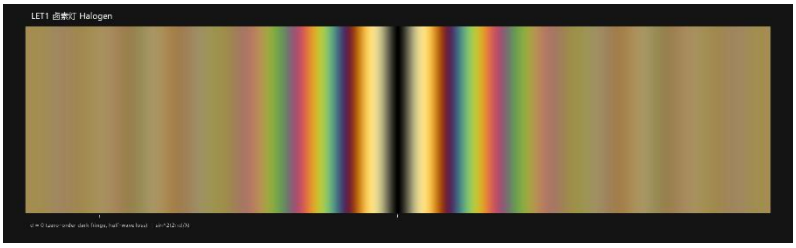
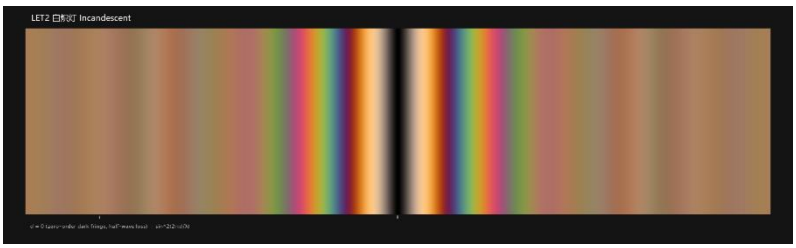


图 3.1.2.1 三类常见白光光源：卤素灯、白炽灯和 LED 灯的光谱分析



(a) 卤素灯



(b) 白炽灯



(c) LED 灯

图 3.1.2.2 三种光源的白光等厚干涉模拟图

光源	光谱特征	干涉条纹表现	综合评价
卤素灯	连续热光谱，峰值约 559 nm，绿、红成分较均衡	颜色过渡自然，但相干长度较短，条纹范围小且整体偏暗	不利于稳定识别
白炽灯	连续热光谱，峰值约 611 nm，色温约 2700 K，红橙成分丰富	中央零级条纹较明显，条纹范围适中，成像亮度和颜色层次较好	选作实验光源
LED 灯	约 452 nm 蓝光芯片叠加 480～730 nm 荧光粉宽带	蓝紫成分突出，相干长度较长、条纹范围较大，中心条纹	不作为首选

光源	光谱特征	干涉条纹表现	综合评价
		辨识较慢	

表 1

综合比较可见：卤素灯的条纹范围较小且亮度偏低；LED 灯虽能形成较宽的条纹区域，但光谱峰值突出、蓝紫成分明显，中心条纹识别耗时较长；白炽灯具有连续光谱，条纹范围、亮度及中心特征较为均衡。为兼顾人工观察、相机成像和 AI 识别稳定性，本系统最终选用白炽灯作为白光扩展光源。

3.1.3 楔角与等厚条纹宽度的定量关系

当定镜 M_1 与动镜虚像 M_2' 之间存在微小夹角 α 时，形成楔形空气膜，产生等厚直条纹。条纹宽度（间距）β 为：

$$\beta = \lambda/2\alpha cos\theta$$

在近正入射条件下($cos\theta \approx 1$)：

$$\beta = \lambda/2\alpha$$

其中$\boldsymbol{\lambda}$为光源波长，$\boldsymbol{\alpha}$为楔角

由此得出条纹宽度随楔角变化规律：α增大，β减小，条纹变密；α减小，β增大，条纹变疏；$\alpha \to 0$，$\beta \to \infty$，过渡为等倾圆条纹。

3.1.4 已知折射率的介质片厚度测量

在白光等厚干涉光路中，插入前两光路光程差为：

$$\Delta L = (2l_2 - 2l_1) + \Delta \tag{2}$$

若在光路中插入一片已知折射率 n 的透明介质片（厚度为 e）此时由于介质片的折射率比空气大，光程发生了改变[4]，插入介质后光程差变化为：

$$\Delta L' = 2[l_2 + (n-1)e] - 2l_1 + \Delta \tag{3}$$

光程差总变化$\delta(\Delta L')$由$\Delta L' - \Delta L$得到等于：

$$\delta(\Delta L') = 2(n-1)e \tag{4}$$

由于介质片引入了额外的光程差，干涉条纹位置将发生移动，中央条纹将移出视场中。启动电机重新调整 M_2'与 M_1 的距离，当中心条纹重新回到视域中央时，由 AI 检测并停止电机转动，由程序读取电机移动距离。此距离即为介质片插入引起的光程差变化量。

由此可根据光程差公式[5]计算出介质厚度：

$$e = \delta(\Delta L')/2(n-1) = 2d'/2(n-1) \tag{5}$$

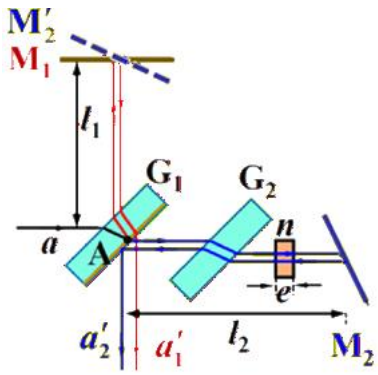


图 3.1.4.1 插入介质片后的干涉示意图

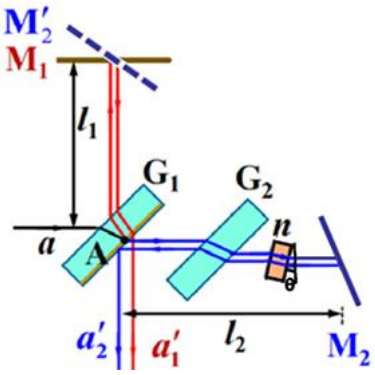


图 3.1.4.2 介质片旋转后的干涉示意图

3.1.5 介质片旋转角度测量

在插入介质片后，若旋转介质片改变其与光轴的夹角 θ，介质片在光路中的有效厚度将发生变化，从而引起光程差的改变，导致干涉条纹再次移动。通过追踪条纹的位移量可以反推出介质片的旋转角度。

根据斯涅耳定律，当光从空气（折射率为 1）进入折射率为 n 的介质时：

$$sin\theta = nsini \tag{6}$$

介质片旋转 θ 角度后，光在介质中的有效路径长度发生变化，光程差总变化为：

$$\delta(L') = 2(\frac{e}{cosi} - e) \tag{7}$$

利用三角恒等式进行化简：

$$sin^2 i + cos^2 i = 1 \tag{8}$$

在一般情况下，旋转角度较小时，$cos\theta \approx 1$，故可进一步简化得到旋转角θ：

$$\theta = n\sqrt{1 - (\frac{2e}{\delta(L')+2e})^2} \tag{9}$$

那么，旋转角与中心条纹移动距离 ΔL 的关系为：

$$\theta = n\sqrt{1 - (\frac{e}{d'+e})^2} \tag{10}$$

3.2 AI 应用原理

3.2.1 YOLOv8 目标检测原理

作为新一代单阶段目标检测模型，YOLOv8 延续了 YOLO 系列"You Only Look Once"的核心设计思想，可通过单次前向传播同步实现目标定位与分类任务。相较于前代模型，其革新性突破在于引入了解耦头（Decoupled Head）结构与 C2f（CSP Bottleneck with two Convolutions）模块。其中，解耦头将分类任务与定位任务分离为两个独立分支，使各分支能够专注于自身任务的特征提取，有效缓解了分类与回归任务之间的冲突，提升了检测精度；C2f 模块则在保留 CSPNet 跨阶段特征复用优势的基础上，通过增加跳跃连接和并行梯度分支，进一步丰富了梯度流信息，在保持轻量化的同时显著增强了多尺度特征表达能力。

YOLOv8 网络由三个核心模块构成：骨干网络（Backbone）、颈部网络（Neck）与预测头网络（Head）。骨干网络的主要功能是从输入图像中提取关键特征信息；颈部网络承担特征融合功能，并将融合结果传输至预测头网络；预测头网络基于解耦结构分别完成目标的分类与定位任务。其结构框架如图 3.2.1.1 所示。

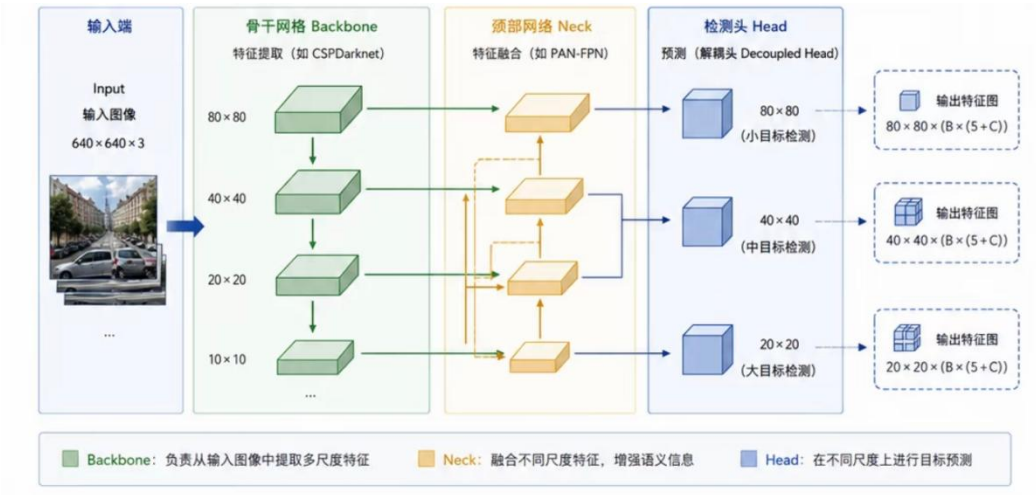


图 3.2.1.1 YOLOv8 结构框架图

为增强特征学习能力，YOLOv8 模型采用了多个高效模块，包括 CBS 模块、C2f 模块及 SPPF（Spatial Pyramid Pooling — Fast）模块。其中，CBS 模块通过整合卷积层、批量归一化及 SiLU 激活函数，实现图像局部特征提取；C2f 模块融合了 CSPNet 与 ELAN 的设计思想，通过多分支梯度流路径进一步提升特征提取性能，尤其

在处理多尺度条纹特征与复杂背景干扰时表现更为稳健；SPPF 模块则通过级联多尺度池化操作，有效扩大感受野，增强模型对全局上下文信息的感知能力。

3.2.2 OCR 读数原理

光学字符识别（Optical Character Recognition，OCR）是将图像中的字符信息转换为机器可处理文本的技术。对于数显微分表，识别对象主要由数字、小数点和正负号构成，目标不是理解自然语言，而是从 LCD 图像中恢复具有固定格式的数值。典型 OCR 流程包括显示区域定位、图像归一化、视觉特征提取、字符序列预测和结果解码。

OCR 系统通常由文本区域处理和字符识别两个环节组成。文本区域处理负责从复杂画面中提取包含字符的有效区域，并通过灰度化、对比度增强和尺寸归一化减小光照、拍摄角度以及显示屏反光造成的差异。字符识别网络则从处理后的图像中提取笔画、边缘和局部形状等视觉特征，将横向排列的字符作为一个完整序列进行识别，因此不需要逐个切分数字。

序列识别方法能够同时利用单个字符的形状信息和相邻字符之间的排列关系。例如，小数点虽然面积较小，但其位置与前后数字共同构成固定的数值格式；负号也可通过其横向结构及所处位置与其他干扰线条区分。识别模型最终输出完整字符序列及相应置信度，为后续数值格式检查和异常结果过滤提供依据。

PP-OCR 系列以轻量化和便于部署为主要特点，通过对特征提取与文本识别网络进行优化，在保持较小计算开销的同时提高字符识别能力。PP-OCRv6 进一步面向检测与识别任务改进网络结构，可覆盖服务器和边缘设备等不同部署场景[6]。项目采用其轻量级文本识别模型，使数显仪表图像能够在本地完成推理，避免依赖云端服务。

OCR 模型输出的字符序列同时带有识别置信度。置信度可用于滤除模糊、反光或遮挡条件下的低质量结果，但单帧高置信度并不必然代表读数真实可靠。LCD 反光、相机抖动、小数点尺寸较小以及数字形状相近，均可能造成字符遗漏或位数错误。因此，仪表读数系统通常还需要结合数值格式约束与连续帧一致性判断，将字符识别结果转换为可信的物理量。

第四章　基于 AI 的辅助系统设计

本系统的核心设计思想是以三大 AI 模块协同辅助白光光学干涉实验：（1）条纹视觉识别——基于 YOLO 神经网络实时识别干涉条纹类别与状态，精确定位中心条纹区域，输出位置与置信度，减少人眼判断差异；（2）数显读数识别——基于 OCR 技术自动识别数显千分尺读数，通过屏幕定位、数字识别和多帧稳定输出，过滤突变、减少读数与抄录误差；（3）AI 实验助手——基于大模型和本地知识库 RAG 驱动，汇总设备状态、判断实验进度，提供预习指导、过程辅助、误差计算和报告生成的全面支持。

4.1 智能体开发

本作品采用流程型智能体架构。流程型智能体按照预先设定的任务流程运行，由流程框架确定任务执行顺序、分支判定条件和数据流转路径。相较于完全自主规划的智能体，该架构能够限制模型的操作范围，使实验指导过程更稳定、可追踪。智能体通过读取实验设备状态和当前任务进度，将知识检索、状态判断和回复生成组织为连续步骤。相关研究表明，大语言模型通过交替执行信息获取、状态观察与推理，可以利用外部环境信息更新任务计划，为智能体与知识库、实验设备等外部模块的协同提供了技术基础[7]。

智能体采用检索增强生成（Retrieval-Augmented Generation，RAG）方法。RAG 将大语言模型的参数化知识与外部知识库结合，在生成回答前检索与问题相关的资料，并将检索内容作为上下文输入语言模型。该方法有助于补充领域知识、提高回答的事实性，同时便于更新和追溯知识来源[8]。本系统的本地知识库存储实验原理、操作步骤、注意事项、误差计算方法及常见问题。学生提出问题后，系统首先检索本地知识库，同时读取设备状态和实验进度，再由大语言模型综合问题、检索结果和实时状态生成回复。

智能体功能覆盖实验全流程的四个环节：预习指导——帮助学生理解实验原理、掌握操作要点；过程辅助——根据当前实验进度和设备状态给出下一步操作建议；误差计算——引导学生完成实验数据处理与不确定度评定；报告生成——汇总实验过程

和测量数据，生成实验报告草稿。已有研究将面向物理实验的专业资料引入大语言模型教学系统，并在大学物理实验课程中验证了其辅助学习的可行性[9]。因此，本系统将大语言模型定位为实验教学辅助工具。



图 4.1.1 AI 智能体功能演示

4.2 视觉方案设计

视觉系统承担三个核心任务：干涉条纹识别、中心条纹识别和微分表视觉读数。进行条纹观察的硬件基础是一台 CCD 工业相机，拍摄镜头参数为：感光度 1/2.8 英寸，有效像素 2592（H）×1944（V）。

4.2.1 干涉条纹识别

干涉条纹识别模块用于判断工业相机画面中是否出现有效的白光干涉条纹，并识别条纹所处的状态，为后续中心条纹精确定位和动镜调节提供候选区域。本系统采用基于 YOLOv8 的目标检测方法，将目标分类与位置回归统一在一次网络推理中，能够同时输出目标类别、边界框和置信度，具有较好的实时性，适用于连续视频画面的在线检测[10]。

工业相机采集原始画面后，系统首先进行画面矫正，并按照预先设定的感兴趣区域（region of interest，ROI）裁剪图像。ROI 限定了干涉图样可能出现的范围，可

排除镜架、光斑外缘及复杂背景的干扰，同时降低模型推理的计算量。裁剪后的图像输入训练完成的 YOLOv8 模型，模型经多尺度特征提取后输出目标类别、检测框及相应置信度，再通过置信度阈值和交并比阈值筛除低质量或重复检测结果。

（1）训练数据构建与标注

结合白光迈克耳孙干涉仪的实际调节过程，训练图像由工业相机在不同动镜位置、曝光条件、条纹清晰度和环境光条件下采集，并从实验视频中间隔抽取具有代表性的图像。数据标注以系统后续控制所需的视觉状态为依据，将近场彩色条纹和中心零级条纹作为主要检测目标。其中，近场彩色条纹表示系统已经进入白光相干范围；中心零级条纹表示零光程差附近的特征条纹已经进入视场。无有效条纹、光斑边缘明显以及图像模糊等画面作为背景或困难样本参与训练，用于降低复杂实验场景下的误检概率。最终构建的数据集包含 350 幅标注图像，其中 280 幅用于模型训练，70 幅用于模型验证。

为避免连续视频中高度相似的相邻帧同时进入训练集和验证集，数据按照采集批次和视频序列进行划分。训练前对标注框进行复核，重点统一中心零级条纹的标注范围，避免因标注尺度差异引入检测框抖动。同时采用适度的亮度、对比度、缩放和平移增强，以模拟曝光波动、相机轻微振动和条纹位置变化。由于条纹颜色本身包含白光干涉状态信息，训练中限制过强的色相与饱和度扰动，以免增强后的样本偏离真实物理图样。

（2）YOLOv8 模型训练

模型采用迁移学习方式进行训练，即在通用目标检测预训练权重的基础上，使用本实验构建的干涉条纹数据集继续优化。模型输入图像尺寸设置为 640×640，批量大小为 16，共训练 100 轮。相较于完全随机初始化，迁移学习能够利用网络已经学习到的边缘、纹理和形状等基础视觉特征，使模型在实验样本数量有限的情况下更快收敛。训练过程中，网络通过边界框回归损失、分类损失和分布焦点损失共同优化目标位置与类别判断，并依据验证集上的综合表现保存最优模型权重。

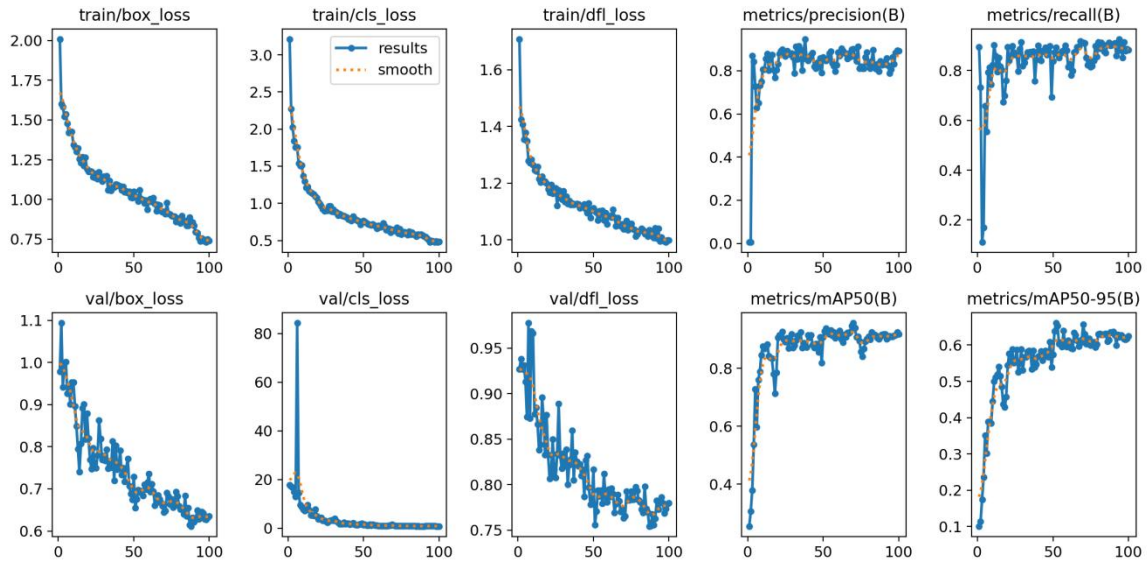


图 4.2.1.1　YOLOv8 模型训练结果曲线

图 4.2.2.1 给出了模型训练过程中损失函数和检测指标的变化。随着训练轮次增加，训练集的边界框损失、分类损失及分布焦点损失整体下降，验证集相关损失也呈下降并逐渐稳定的趋势，表明模型逐步学习到干涉条纹的类别和位置特征。Precision、Recall 及 mAP 指标在训练前期快速提升，随后在一定范围内趋于稳定，说明模型训练已经基本收敛。训练结束时，模型在验证集上的 Precision 为 0.892，Recall 为 0.883，mAP@0.5 为 0.918，mAP@0.5:0.95 为 0.624，表明模型能够较稳定地完成干涉条纹目标识别与候选区域定位。曲线中的局部波动主要与验证样本数量、条纹形态差异及曝光变化有关，因此模型选择不以单个轮次的峰值为依据，而综合考虑验证指标、连续轮次稳定性及实际视频检测效果。

（3）条纹状态判定与连续帧处理

模型部署后，当检测结果中不存在满足置信度阈值的条纹目标时，系统将当前画面判定为无有效条纹状态，并继续按照既定方向驱动动镜搜索；检测到近场彩色条纹后，表明系统进入白光相干范围，控制程序降低搜索速度并加强对中心零级条纹的检测；当中心零级条纹稳定出现时，其检测框作为后续精确定位算法的候选区域。由此形成“图像感知—状态识别—候选区域提取—调节控制”的处理链路。

针对动镜运动、机械振动和光强变化造成的条纹位置漂移，本系统利用目标检测

模型的实时性持续更新候选区域。但需要指出，YOLO 检测框反映的是中心零级条纹可能存在的区域，其几何中心会受到标注尺度、条纹宽度、目标姿态和帧间抖动的影响，不能直接等同于物理中心条纹的位置。因此，YOLO 模块主要承担“有无条纹、条纹类别及大致位置”的粗检测任务；检测到中心零级条纹后，再由图像特征融合算法在候选框内完成横向坐标的精确定位。

为提高视频检测的稳定性，系统对连续帧结果进行一致性判断。只有当目标类别和位置在连续若干帧内保持稳定时，才确认条纹状态发生变化；对于单帧低置信度结果或短时漏检，暂时保留上一帧的有效状态。该处理能够减少曝光波动、运动模糊和偶发误检对电机控制过程的影响，并为中心条纹精确定位提供连续、可靠的候选区域。

4.2.2 中心条纹识别

中心条纹识别模块在 YOLO 给出的中心零级条纹候选区域内进一步确定中心条纹的横向坐标。白光干涉条纹由多个波长的干涉结果叠加形成，在零光程差附近具有较强的局部振荡能量和近似左右对称的相干包络。傅里叶条纹分析和小波分析研究表明，去除缓慢变化的背景后，可利用条纹的局部频率、振荡能量和包络特征提取有效干涉信息[11-12]。基于这一思想，本项目结合实际图像中中心条纹可能呈暗、亮或带色的特点，构建多特征加权评分函数，而不是仅搜索画面中最暗的一条条纹。

首先，将候选区域的 B、G、R 三个颜色通道分别沿条纹方向进行平均，得到随横向位置 x 变化的一维颜色强度序列。随后采用高斯平滑估计缓慢变化的照明背景，并用原始序列减去背景得到高通分量。高通分量保留条纹的周期性明暗变化，可据此计算局部条纹能量、梯度能量以及相干包络。系统再将亮度、色度、条纹能量、边缘结构和左右对称性等特征归一化至[0,1]区间，形成中心位置综合评分：

$$Q(x)=0.25D(x)+0.30C(x)+0.16S(x)+0.12E(x)+0.10M(x)+0.03P(x)+0.04B(x) \quad (11)$$

式中各项权重为项目根据验证图像上的定位误差和连续帧稳定性进行调整后采用的经验权重，用于反映不同特征对实际中心定位的相对贡献，并非 YOLO 网络的训练参数。各特征的含义如下。

符号	特征名称	含义与作用
D(x)	暗度特征	表示位置 x 相对于局部背景的暗程度，用于增强呈暗纹状态的中心候选。
C(x)	色度特征	描述 RGB 三通道在位置 x 的离散程度，用于反映白光彩色条纹的颜色变化。
S(x)	条纹能量	表示去除低频照明背景后残余振荡的强度，用于判断局部是否存在清晰、周期性的干涉条纹。
E(x)	对称边缘对	判断候选条纹左右两侧是否存在方向相反、位置近似对称的边缘。
M(x)	包络对称性	衡量位置 x 左右两侧条纹能量包络的镜像相似程度。
P(x)	YOLO 位置先验	依据 YOLO 零级条纹检测框构造弱高斯先验，用于限定搜索范围，但不强制中心位于检测框几何中心。
B(x)	边界抑制	抑制紧贴检测框边缘的伪结构，降低框线、光斑边界或裁剪边缘造成的误定位。

在 YOLO 检测框限定的有效区间内，取综合评分最大的整数像素位置作为中心条纹的初始位置：

$$\hat{x}=\arg\max_{x} Q(x) \tag{12}$$

由于相机图像为离散采样，直接取最大值只能得到整数像素坐标，连续帧中容易出现一个像素范围的跳动。为提高定位分辨率，系统使用峰值及其左右相邻三个评分点进行抛物线插值：

$$x^{*}=\hat{x}+\frac{1}{2}\cdot\frac{Q(\hat{x}-1)-Q(\hat{x}+1)}{Q(\hat{x}-1)-2Q(\hat{x})+Q(\hat{x}+1)} \tag{13}$$

该公式将评分峰值邻域近似为二次曲线，以曲线顶点作为修正后的亚像素位置 x^{*}。二次多项式拟合已被用于白光干涉条纹峰值的精细定位，能够在保持较低计算量的同时减小离散采样引起的定位误差[13]。当分母过小或插值偏移超过半个像素时，系统限制修正量，以防止平坦峰值或噪声造成异常结果。

最后，系统对连续帧的中心位置进行跟踪和平滑。首先对最近若干帧的有效坐标取中位数，以抑制偶发误检；再将该中位数与上一帧输出加权融合。若新检测位置相对历史位置发生超过设定阈值的突变且置信度不足，则拒绝该帧结果；短时漏检时保持上一有效位置并逐帧降低置信度，连续漏检超过限定帧数后重新进入搜索状态。得

到稳定中心坐标后，计算中心条纹与视场中心之间的横向偏差 e=x*−xview。当|e|小于允许误差并连续若干帧满足条件时，系统判定中心条纹已对准，停止电机并记录微分表位置。

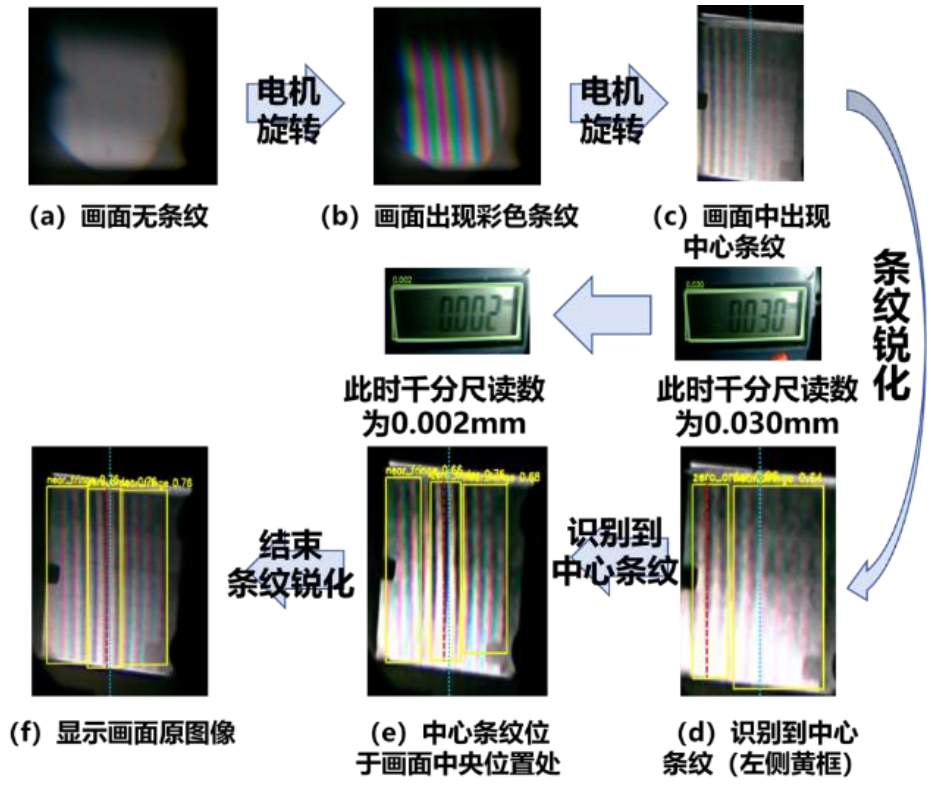


图 4.2.2.1 识别白光干涉中心条纹的全流程

4.2.3 微分表视觉读数

微分表视觉读数模块使用独立 USB 相机采集数显微分表的 LCD 画面，并按照“显示区域定位—图像增强—OCR 识别—连续帧确认”的流程输出读数。采用独立相机可以避免干涉条纹图像采集与仪表读数相互影响，并使两路视觉任务能够同时运行。

系统首先根据 LCD 屏幕的长条形轮廓和矩形结构，在相机画面中提取显示区域；自动定位失败时，可使用预先设置的感兴趣区域。随后对裁剪图像进行灰度化和限制对比度自适应直方图均衡化，以增强数字与背景之间的局部对比度并减弱光照不均的影响[14]。

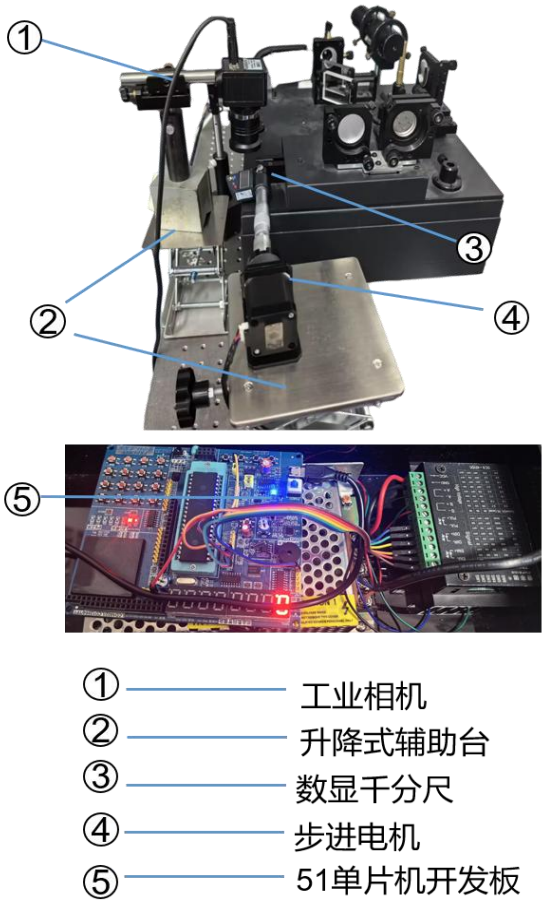
处理后的 LCD 图像输入 RapidOCR 调用的 PP-OCRv6 轻量级识别模型，得到数字、小数点和正负号组成的字符序列及置信度[6]。系统对识别文本进行数值格式检查，并对小数点遗漏等常见问题进行有限修正；格式错误或置信度过低的结果不参与读数更

新。

为降低相机抖动、屏幕反光和单帧误识别的影响，系统对连续多帧结果进行一致性确认，只有读数连续稳定出现后才作为最终输出。稳定读数以毫米为单位提供给实验流程，在中心条纹对准并停止电机后自动记录，用于计算动镜位移。该方法减少了人工观察和抄录造成的误差，也避免偶发 OCR 结果直接影响电机控制。

4.3 动镜辅助调节系统设计

动镜辅助调节系统由四个核心部件组成：①工业相机——用于拍摄数显千分尺的 LCD 读数屏幕；②升降式辅助台——用于搭载和固定相机及调节装置；③数显千分尺——替代传统机械千分尺，以数字显示动镜位移量；④51 单片机开发板和步进电机——实现电机的驱动和控制。



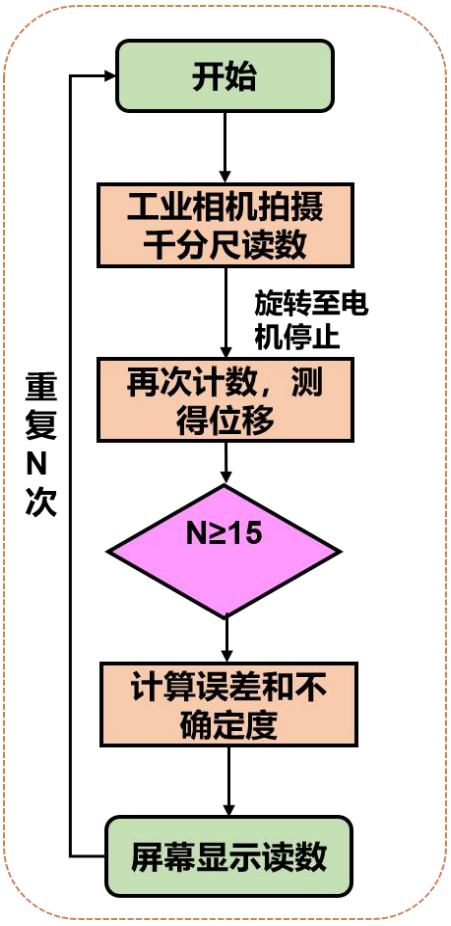


图 4.3.1 动静辅助调节系统图片　　图 4.3.2 动静辅助调节系统工作流程

系统的工作流程为：实验开始时，工业相机拍摄数显千分尺的当前读数作为初始位置；电机驱动千分尺旋转带动动镜移动，视觉系统持续追踪中心条纹位置；当中心条纹到达目标位置后，电机自动停止；系统再次计数并测得位移量；该过程重复 N 次（N≥15），以确保测量可靠性；最后计算误差和不确定度，并在屏幕上显示测量结果。

在效果展示中，系统实现了实时检测条纹特征、识别到目标条纹时自动停止电机转动的功能。视觉识别结果通过红色虚线标记中央条纹位置，蓝色虚线标记视场中央位置，当二者重合时说明中心条纹已对准。

第五章　测量精度与不确定度

5.1 实验误差分析

本实验的测量目标是评定基于计算机视觉的迈克尔逊白光等厚干涉条纹定位精度和介质厚度测量精度。系统通过对中心条纹的自动识别与追踪，结合数显千分尺的位移读数，实现对干涉条纹位置的精确标定。测量过程中的误差来源主要包括系统误差和随机误差两大类。

1. 系统误差

（1）视觉定位误差

YOLO 神经网络模型对中心条纹位置的检测置信度存在波动。在连续帧检测中，检测框中心位置因光照变化、条纹对比度波动和模型推理随机性而产生像素级抖动。由于白炽灯光源的强度可能存在微小波动，CCD 相机在不同帧的曝光均匀性也会引入图像噪声，这些因素共同导致视觉定位的不确定性。

误差控制措施：采用连续多帧跟踪与平滑滤波算法，抑制单帧检测框的瞬时抖动。选取固定水平截线上的暗纹光强极小值作为条纹位置的补充判据，降低模型置信度波动的影响。

（2）机械传动误差

步进电机与数显千分尺之间存在传动间隙。在电机驱动千分尺旋转过程中，联轴器的弹性变形和齿轮啮合间隙会导致回程差。此外，数显千分尺内部丝杆的螺距误差和空程也会引入系统性的位置偏差。

误差控制措施：所有初始位置和终止位置的逼近均严格从同一机械方向进行，使回程间隙的影响进入完整测量过程的重复性中，避免正反向差异。接近目标位置时降低电机转速，减小惯性过冲。

（3）OCR 读数误差

数显千分尺 LCD 屏幕在特定角度下可能产生反光或字符模糊，导致 OCR 引擎出现字符误识别。光照条件变化和相机对焦偏差也会影响读数精度。

误差控制措施：采用多帧确认机制，对连续若干帧的 OCR 识别结果进行投票，取稳定一致的读数作为最终输出，有效滤除偶发性误识别。

（4）环境因素

环境振动、空气扰动和温度漂移会对迈克尔逊干涉仪光路稳定性产生扰动，引起干涉条纹的缓慢移动或抖动。白光的相干长度仅为微米量级，对环境变化较为敏感。

误差控制措施：实验在隔振光学平台上进行，实验前对激光器和电子位移计进行充分预热（10～20 min），保证测量环境相对安静。利用高通滤波算法滤除环境引起的低频大幅值相位扰动。

2. 随机误差

随机误差主要来源于测量过程中一系列因素微小的、不可控的随机波动，包括条纹定位的帧间抖动、数显读数的末位跳变以及操作过程中无法完全消除的随机干扰。

误差控制措施：通过多次重复测量（N ≥ 15）取平均值的方法来降低随机误差的影响，以测量列的标准差作为随机误差的量化指标。

5.2 条纹定位精度的不确定度测量

5.2.1 测量模型

依据不确定度的评定标准构建测量模型[14-16]。本实验的被测量为两次条纹定位之间的位移量 d，测量模型如下：

$$d = \overline{d} + \delta_{cal} + \delta_{stepper} \tag{14}$$

其中：

$\overline{d}$ —— 初始位置与终止位置的数显千分尺视读位移之差 $\overline{d} = \frac{1}{n}\sum_{i=1}^{n} d_i$，由重复测量统计评定（A 类）；

δ_{cal} ——数显千分尺的校准修正值（B 类）；

$\delta_{stepper}$ ——步进电机的位移精度值（B 类）；

5.2.2 测量方法与数据

本次测量采用追踪中心条纹的方式：选择电机转动前千分尺的读数为初始位置，以电机停转时条纹到达视场中央时为终止位置。每次实验开始前将千分尺转回初始位置的读数大小，实验过程中从同一电机转动方向接近终止位置，用数显千分尺读数记录终止处的稳定位置。每组实验在终止位置读 2 次数取平均，独立重复 5 组完整位移过程。

依据测量方案的要求，正式实验中对每个完整位移过程独立重复 N = 15 次（或按照实际实验数据确定），以保证统计分析的可靠性

测量结果如下表所示：

组号i	初始位置两次读数(mm)		平均初始位置 (mm)	终止位置两次读数(mm)		平均终止位置 (mm)	位移d_i (mm)
1	0.030	0.030	0.030	0.002	0.002	0.002	0.028
2	0.030	0.030	0.030	0.002	0.002	0.002	0.028
3	0.030	0.030	0.030	0.003	0.003	0.003	0.027
4	0.030	0.030	0.030	0.004	0.003	0.004	0.026
5	0.030	0.030	0.030	0.004	0.004	0.004	0.026

表 2 条纹定位精度测量数据表

5.2.3 标准不确定度的 A 类评定

A 类标准不确定度通过对重复测量数据的统计分散性进行评定。

对 N 次完整位移测量结果构成测量列$\{d_1, d_2, \ldots, d_N\}$，位移的平均值为：

$$\overline{d} = \frac{1}{n}\sum_{i=1}^{n} d_i \tag{15}$$

测量列的实验标准差为：

$$S_d = \sqrt{\frac{\sum_{i=1}^{n}(d_i-\overline{d})^2}{n-1}} \tag{16}$$

对应的不确定度为：

$$u_A = \frac{S_d}{\sqrt{n}} \tag{17}$$

5.2.4 标准不确定度的 B 类评定

（1）数显千分尺示值误差引入的不确定度

本实验采用数显千分尺测量动镜位移。根据仪器说明，数显千分尺的最大允许误

差（MPE）为 $\Delta_{MPE} = 0.003\,mm$

数显千分尺的测量误差来源于机械结构的随机磨损、操作时的微小倾斜和测力波动等随机因素，这些因素使误差大概率在期望值附近波动，中间值概率高、极端值概率低，符合正态分布的特征。因此按正态分布处理，置信因子取 $C = 3$：

$$u_{cal} = \frac{\Delta_{MPE}}{3} \tag{18}$$

（2）步进电机位移精度引入的不确定度

本实验采用步进电机带动数显千分尺转动。由仪器给定的最小步进角度算得误差为$\Delta_2 = 0.0025mm$

步进电机的步距角精度由制造商以最大允差给出，仅知误差上下限而无具体分布信息，采用等分假设——取矩形分布，置信因子$C = \sqrt{3}$:

$$u_{stepper} = \Delta_2/\sqrt{3} \tag{19}$$

5.2.5 合成标准不确定度

各不确定度分量相互独立时，合成标准不确定度 $u_C(d)$ 由各分量按 RSS（Root Sum of Squares）方法合成：

$$u_C = \sqrt{{u_A}^2 + {u_{cal}}^2 + {u_{stepper}}^2} \tag{20}$$

5.3 介质厚度测量与不确定度分析

5.3.1 测量模型

同样，依据不确定度的评定标准构建测量模型。本实验的被测量为介质厚度 e，测量模型如下：

$$e = \frac{d}{(n-1)} \tag{21}$$

其中：

d — 5.2 中测得两次条纹定位之间的位移量，对 d 灵敏系数：$C_d = \frac{\partial e}{\partial d} = \frac{1}{n-1}$ ；

n — 介质折射率，灵敏系数$C_n = \frac{\partial e}{\partial n} = -\frac{\bar{e}}{n-1}$ ；

5.3.2 测量方法与数据

本次测量采用追踪中心条纹的方式：在实验开始前先将中心条纹调至视场中心位置，并用数显千分尺记录初始位置，在动镜光路加入透明介质薄片，干涉条纹消失在视场中，实验过程中使电机沿同一方向转动，直至识别到画面中有条纹出现，左右自动搜寻，直至识别到透过薄片的中心条纹再次位于视野中央，用数显千分尺读数记录终止处的稳定位置。每组实验在初始与终止位置读 2 次数取平均，独立重复 5 组完整位移过程。依据测量方案的要求，正式实验中对每个完整位移过程独立重复 N = 15 次（或按照实际实验数据确定），以保证统计分析的可靠性

测量结果如下表所示：

序号	1	2	3	4	5	平均值
动镜位移 d/ mm	0.09120	0.09160	0.09250	0.09140	0.09270	0.09188
厚度 e/ mm	0.175385	0.176154	0.177885	0.175769	0.178269	0.176692
厚度 e/ μm	175.385	176.154	177.885	175.769	178.269	**176.692**

表 3 介质厚度测量数据表

5.3.3 标准不确定度的 A 类评定

千分尺端对应的自由度为：

$$u_A = \frac{S_d}{\sqrt{n}} \tag{22}$$

传播到厚度：

$$u_A(e) = C_d u_A(\overline{d}) \tag{23}$$

5.3.4 标准不确定度的 B 类评定

(1)数显千分尺示值误差引入的不确定度:

$$u_{cal} = \frac{\Delta_{MPE}}{3} \tag{24}$$

传播到厚度：

$$u_{cal}(e) = C_d u_{cal}(\overline{d}) \tag{25}$$

(2)步进电机位移精度引入的不确定度:

$$u_{stepper} = \Delta_2/\sqrt{3} \tag{26}$$

传播到厚度：

$$u_{stepper}(e) = C_d u_{stepper}(\overline{d}) \tag{27}$$

(3)盖玻片折射率精度引入的不确定度：

依据柯西色散公式 $n(\lambda) = A + B/\lambda^2$ 在可见光窄波段内中心最可能、两端递减的特征，分布取三角分布。设定半宽度为 a。

$$u_{(n)} = a/\sqrt{6} \tag{28}$$

传播到厚度：

$$u_{(n)}(e) = C_n u_{(n)}(\overline{d}) \tag{29}$$

5.3.5 合成标准不确定度

各不确定度分量相互独立时，合成标准不确定度 $u_C(e)$ 由各分量按RSS（Root Sum of Squares）方法合成：

$$u_C(e) = \sqrt{u_A(e)^2 + u_{cal}(e)^2 + u_{stepper}(e)^2 + u(n)(e)^2} \tag{30}$$

5.4 微小转角测量与不确定度分析

5.4.1 测量模型

实验在定镜 M_1与分光板 G_1之间放置角位移台，台上固定一片厚度 0.3 mm、折射率 1.46 的均匀石英玻璃片。以垂直光轴方向为水平方向，测量介质片相对水平方向的微小转角，实验分为两步：手动调节角位移台，将介质片从 8° 转动至 10°；运行 AI 程序，驱动电机转动，让偏移的中心黑条纹重回视场中央，读取千分尺的示数变化，代入公式算出转角，再和角位移台的人眼读数对比。测量结果如下表所示：

序号	1	2	3	4	5	平均值
动镜位移 d/ mm	0.011	0.012	0.011	0.011	0.013	0.0116
转角 Δθ/ °	1.88	2.03	1.88	1.88	2.18	1.97

表 4 微小转角测量数据表

依据不确定度的评定标准构建测量模型[14-16]。本实验的被测量为介质片相对水平方向的微小转角$\Delta\theta$。由 3.1.5 节的测量原理，介质片旋转后光程差的改变由动镜位移补偿，将千分尺示数变化代入公式即可算得转角。由表中数据，转角与动镜位移近似满足线性关系，测量模型如下：

$$\Delta\theta = C_d d \tag{31}$$

其中：

d —— 千分尺移动的距离（mm）；$\Delta\theta$ —— 介质片微小转角（°）；C_d —— 转角对动镜位移的灵敏系数，由表中测量数据评定：

$$C_d = \frac{2.18-1.88}{0.013-0.011} = 150°/mm \approx 2.62 rad/mm \tag{32}$$

5.4.2 标准不确定度的 A 类评定

A 类标准不确定度通过对重复测量数据的统计分散性进行评定。对 5 次完整测量结果构成测量列，动镜位移的平均值为：

$$\overline{d} = \frac{1}{N}\sum_{i=1}^{N} d_i = 0.0116\ mm \tag{33}$$

测量列的实验标准差为：

$$S_d = \sqrt{\frac{\sum_{i=1}^{N}\left(|d_i| - |\overline{d}|\right)^2}{N-1}} = 0.000894\ mm \tag{34}$$

对应的不确定度为：

$$u_A(d) = \frac{S_d}{\sqrt{N}} = 0.0004\ mm \tag{35}$$

传播到转角：

$$u_A(\Delta\theta) = C_d u_A(d) = 150 \times 0.0004 = 0.060° \tag{36}$$

A 类分量的自由度为$\nu_A = N - 1 = 4$。对表中 5 个转角结果直接统计可得$s(\Delta\theta) = 0.134°$、$u_A(\Delta\theta) = 0.060°$，与传播结果一致。

5.4.3 标准不确定度的 B 类评定

（1）数显千分尺示值误差引入的不确定度

本实验采用数显千分尺测量动镜位移。根据仪器说明，数显千分尺的最大允许误差（MPE）为$\Delta_{MPE} = 0.003\ mm$。按正态分布处理，置信因子取$C = 3$：

$$u_{cal}(d) = \frac{\Delta_{MPE}}{3} = 0.001\ mm \tag{37}$$

传播到转角：

$$u_{cal}(\Delta\theta) = C_d u_{cal}(d) = 150 \times 0.001 = 0.15° \tag{38}$$

（2）步进电机位移精度引入的不确定度

本实验采用步进电机带动数显千分尺转动。由仪器给定的最小步进角度算得误差为$\Delta_2 = 0.0025\ mm$。取矩形分布，置信因子取$C = \sqrt{3}$：

$$u_{step}(d) = \frac{\Delta_2}{\sqrt{3}} = 0.00144\ mm \tag{39}$$

传播到转角：

$$u_{step}(\Delta\theta) = C_d u_{step}(d) = 150 \times 0.00144 = 0.216° \approx 0.22° \tag{40}$$

石英玻璃片的厚度 e = 0.3 mm 与折射率 n = 1.46 采用实验给定标称值，其不确定度未在实验材料中给出，本次评定未将其计入合成。

5.4.4 合成标准不确定度

各不确定度分量相互独立时，合成标准不确定度$u_C(\Delta\theta)$由各分量按 RSS（Root Sum of Squares）方法合成：

$$u_C(\Delta\theta)=\sqrt{\left(u_A(\Delta\theta)^2\right|+\left|u_{cal}(\Delta\theta)^2\right|+\left|u_{step}(\Delta\theta)^2\right)}=\sqrt{(0.060^2|+|0.15^2|+|0.216^2)}\approx 0.27^\circ \tag{41}$$

微小转角测量结果为$\Delta\theta=(1.97\pm0.27)^\circ$，相对合成标准不确定度约为 14%。与人眼直接读数相比，该测量方法精度更高，更具可信度。

5.5 彩色白光干涉的盖玻片厚度起伏辅助分析

5.5.1 理论分析

若透明片替代同厚度空气层、近正入射，局部厚度 t 与附加光程差 A 满足：

$$A=(n-1)t,\quad t=\frac{A}{(n-1)} \tag{31}$$

白光是复色光，彩色相机的 R、G、B 通道各有不同的中心波长和相干包络。对通道 m，在沿光程方向扫描时，干涉强度可以表示为背景光强与包络调制的振荡项之和：

$$I_m(z)=I_{0m}[1+\gamma_m g_m(z-z_0)\cos(\frac{4\pi(z-z_0)}{\lambda_{c,m}})] \tag{32}$$

$$g_m(z-z_0)=\exp[-\left(\frac{z-z_0}{l_{c,m}}\right)^2] \tag{33}$$

式中 z_0 为零光程差位置，γ_m 为条纹可见度，g_m 由通道光谱决定。只有在 z_0 附近，相干包络和条纹对比度才较高，通道颜色与条纹级次因而携带局部光程差变化的信息。三通道扫描相位还可以联合确定零光程差位置，减小离散扫描峰值与真实零点的偏差。

$$\hat{z}_0=-\frac{\sum_{m\in\{R,G,B\}}A_m B_m}{\sum_{m\in\{R,G,B\}}A_m^2},\phi_m(z)=A_m z+B_m \tag{34}$$

上述扫描模型给出的是彩色信息用于形貌分析的理论参照[1]。单帧图像不含沿光

程方向的扫描序列，且颜色受光源谱、相机响应与白平衡影响，需先经 5.5.2 的颜色—光程差标定把颜色映射为附加光程差。

5.5.2 最终项目中的程序功能

厚度分布功能位于项目主程序中。用户在“05 临时测量—薄膜厚度分布（单帧）”面板选择干涉画面和感兴趣区（ROI），设置波长、折射率，可选输入无膜基准图和颜色—光程差标定文件。界面调用位于 src/ui/app.py，厚度分布计算位于 src/vision/thickness_distribution.py。

程序在 Lab 色彩空间把标定点依次连成分段折线，每个像素投影到折线的线段上取对应光程差。白光颜色随光程差周期性重复，同一颜色可能对应多个候选值，因此程序同时计算条纹的空间解包相位，用相邻像素光程差连续这一条件在候选中选取；正负两个符号分支各解一次，取整幅图匹配更好的一支。

每个像素还输出一个 0—1 的置信度，由四项加权得到：与标定折线的 Lab 残差（0.45）、所选分支与解包相位的一致性（0.25）、条纹本身的色度与纹理质量（0.25）、最佳与次佳候选的可分辨程度（0.05）。置信度经 1 px 高斯邻域归一化，用于抑制单像素噪声。

程序可输出伪彩叠加图、二维厚度热图，以及有效像素数、稳健峰谷值、RMS 起伏等统计量；输入无膜基准图时对背景作扣除。厚度锚点只平移整幅图的显示零点，不校正局部级次错误和颜色模型偏差。

$$t_{\text{display}}(x,y) = t(x,y) - \text{median}(t) + t_0 \tag{35}$$

标定模式下厚度按 $t = \frac{A}{(n-1)}$ 换算，A 为标定文件给出的附加光程差。

5.5.3 示例结果

以项目示例彩色干涉图像为输入，人工框选样品区域作为 ROI，取 x=103、y=46、宽 145 px、高 151 px（坐标自 0 起），该区域避开夹具、暗背景和样品边缘。分析参数为波长 589.3 nm、折射率 1.523、平滑 2 px。ROI 内 145×151 = 21 895 个像素全部参与统计。厚度基准 170.0 μm 取自拍摄该图时按 5.3 节方法测得的样品厚度。

输出指标	结果
有效像素	21 895
稳健峰谷值（P98-P2）	1.323 μm
相对均值 RMS	0.432 μm
中央 90% 厚度跨度（P95-P5）	1.252 μm
综合置信度中位数	0.596

表 5.5.1 示例图 ROI 的厚度统计结果

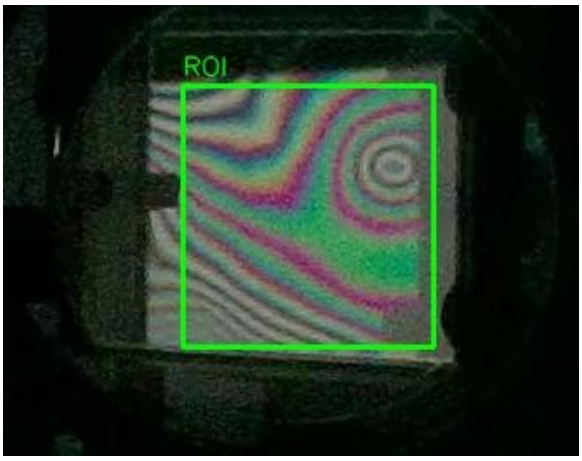


图 5.5.1 原始彩色干涉图与人工标注的 ROI

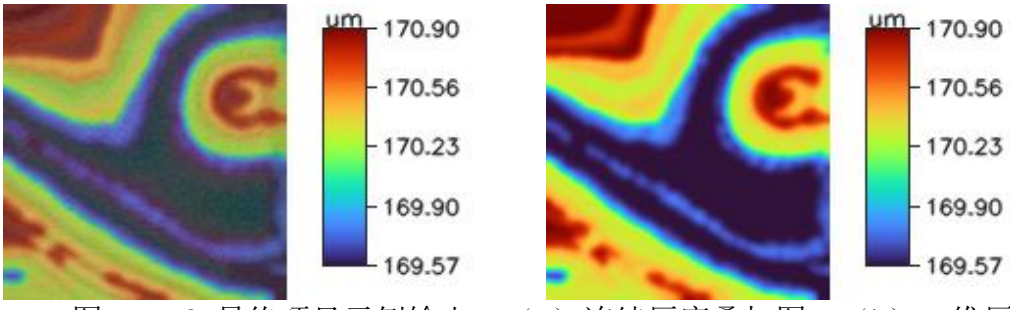


图 5.5.2 最终项目示例输出：（a）连续厚度叠加图；（b）二维厚度热图

条纹弯曲且局部疏密不同，厚度图相应出现非均匀起伏。稳健峰谷值 1.323 μm 是厚度分布第 98 与第 2 百分位之差，RMS 0.432 μm 是各像素相对均值的偏离，二者分别约为参考厚度的 0.78 % 和 0.25 %。中央 90% 跨度 1.252 μm 与稳健峰谷值接近，说明起伏在 ROI 内分布比较均匀。置信度中位数 0.596，属于中等水平。

5.5.4 不确定度传播

5.5 与 5.3 使用同一测量模型：厚度 t 由附加光程差 A 和折射率 n 决定，$t = \frac{A}{(n-1)}$。对 A 和 n 的灵敏系数与合成标准不确定度为：

$$C_A = \frac{1}{(n-1)}, \quad C_n = -\frac{t}{(n-1)} \tag{36}$$

$$u_c(t)=\sqrt{\left(\frac{u(A)}{(n-1)}\right)^2+\left(t\frac{u(n)}{(n-1)}\right)^2} \tag{37}$$

附加光程差 A 的标准不确定度由颜色—光程差标定的分辨力决定：标定表相邻读数对应 0.05 μm 的光程差，按矩形分布取半宽 0.025 μm，得 $u(A)=\frac{0.025}{\sqrt{3}}$ = 0.014 μm。

折射率 n 的标准不确定度沿用 5.3.4 的评定：折射率允差 ±0.001 按三角分布取半宽，得 $u(n)=\frac{0.001}{\sqrt{6}}$ = 0.00041。

代入 t = 170.0 μm、n = 1.523，得 C_A = 1.91、C_n = −325.0 μm，$u_c(t)$ = 0.14 μm。取包含因子 k = 2，扩展不确定度 U = 0.27 μm，相对值 0.16 %。

第六章　实验创新点与总结

6.1 实验创新点

创新点一：智能体开发。

本项目创新研发实验辅助智能体，可贯穿实验全部流程，提供过程化引导、动态实时反馈与个性化评测，有效解决实验教学中的现实难题。智能体基于大语言模型和 RAG 本地知识库，能够在实验预习、过程辅助、误差计算和报告生成等环节提供精准、及时的指导，减轻教师重复性答疑负担。

创新点二：视觉辅助系统。

基于 YOLO 神经网络和工业相机，实现了白光干涉中心条纹的自动识别与定位。视觉系统能够实时捕捉干涉条纹图样，自动判断条纹类型（近场彩色条纹、中心零级条纹），并以红蓝双色虚线分别标记中央条纹位置和视场中央位置，直观展示调节效果。同时，OCR 读数识别模块实现了数显千分尺的自动阅读，通过多帧稳定确认消除人工读数误差。

创新点三：动镜调节辅助系统。

装置自主搭建融合 C51 单片机、数显千分尺、步进电机的动镜调节辅助系统，有效降低白光干涉实验操作门槛，实现实验过程的信息化与自动化。该系统无需对迈克尔逊干涉仪本体进行结构改动，安装和拆卸便捷，适合在现有教学实验室中推广部署。

6.2 总结

本作品针对迈克尔逊干涉仪白光等厚干涉实验的教学痛点，以命题六"AI+物理实验"的要求为指引，研制了一套集智能体辅助教学、计算机视觉条纹识别、电机自动调节于一体的智能辅助实验系统。作品遵循"基于特定物理原理、AI 技术深度融合、适用于教学场景"三项设计目标，将大语言模型、YOLO 视觉检测、OCR 读数识别和单片机电机控制等技术有融合于经典物理实验中。

系统的硬件成本约 1200 元（不含迈克尔逊干涉仪本体和计算机），在保持经济性的同时实现了实验效率的显著提升，为大学物理实验教学的数字化转型和 AI 赋能提供了可行的技术方案。

第七章　参考文献

[1] 李劲松. 基于白光和激光干涉的高灵敏微位移测量技术研究[D]. 武汉: 湖北工业大学, 2020.

[2] 彭真真, 赵硕洽, 刘月林, 程匹克, 陈程. 非接触法自动测量薄膜厚度[J]. 大学物理实验, 2011, 24(4): 33-36. DOI: 10.14139/j.cnki.cn22-1228.2011.04.004.

[3] 乔亮. 迈克尔逊干涉实验的仿真研究[J]. 重庆三峡学院学报, 2015, 31(3): 50-52. DOI: 10.13743/j.cnki.issn.1009-8135.2015.03.014.

[4] 鄢仁文, 周汉枢. 迈克尔逊干涉测薄膜厚度[J]. 福建师范大学福清分校学报, 2009(S1): 43-46.

[5] 窦玲玉, 姬超燃, 任立庆, 李洪国, 宋新兵. 迈克耳孙干涉仪白光干涉现象分析[J]. 大学物理, 2024, 43(9): 50-53. DOI: 10.16854/j.cnki.1000-0712.230353.

[6] ZHANG Y, WANG X, LIN M, et al. PP-OCRv6: From 1.5M to 34.5M Parameters, Surpassing Billion-Scale VLMs on OCR Tasks[EB/OL]. arXiv:2606.13108, 2026.

[7] YAO S, ZHAO J, YU D, et al. ReAct: Synergizing Reasoning and Acting in Language Models[C]//International Conference on Learning Representations. 2023.

[8] LEWIS P, PEREZ E, PIKTUS A, et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks[C]//Advances in Neural Information Processing Systems. 2020, 33: 9459-9474.

[9] 姜文印, 刘宸宇, 王宇兴, 等. 大语言模型在大学物理实验课程中的应用探究[J]. 大学物理, 2025, 44(7): 36-. DOI: 10.16854/j.cnki.1000-0712.240389.

[10] REDMON J, DIVVALA S, GIRSHICK R, et al. You Only Look Once: Unified, Real-Time Object Detection[C]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition. 2016: 779-788.

[11] TAKEDA M, INA H, KOBAYASHI S. Fourier-transform method of fringe-pattern analysis for computer-based topography and interferometry[J]. Journal of the Optical Society of America, 1982, 72(1): 156-160. DOI: 10.1364/JOSA.72.000156.

[12] SANDOZ P. Wavelet transform as a processing tool in white-light interferometry[J]. Optics Letters, 1997, 22(14): 1065-1067. DOI: 10.1364/OL.22.001065.

[13] PARK M C, KIM S W. Direct quadratic polynomial fitting for fringe peak detection of white light scanning interferograms[J]. Optical Engineering, 2000, 39(4): 952-959. DOI: 10.1117/1.602445.

[14] PIZER S M, AMBURN E P, AUSTIN J D, et al. Adaptive histogram equalization and its variations[J]. Computer Vision, Graphics, and Image Processing, 1987, 39(3): 355-368.

[15] 测量不确定度评定和表示: GB/T 27418—2017[S]. 北京: 中国标准出版社, 2018.

[16] International Organization for Standardization. Guide to the expression of uncertainty in measurement: ISO/IEC Guide 98-3:2008[S]. Geneva: ISO, 2008.

[17] 测量不确定度评定与表示: JJF 1059—2012[S]. 北京: 中国计量出版社, 2012.

附录一：成本清单

设备	数量	价格（元）
CCD 相机	1 台	200
光学镜头	1 个	158
步进电机	1 个	50
C51 单片机	1 块	89
数字化千分尺	1 个	496
电机驱动+电源	1 套	76
AI 部署	—	100
总费用	—	约 1200 元

附录二：成员分工

成员	分工
成员 1	实验装置的搭建，代码编写，PPT 制作，视频录制
成员 2	实验数据的获取，步进电机的搭建，论文撰写，视频录制
成员 3	装置搭建，实验数据的获取，误差分析，不确定度计算

附录三：实验选用工业相机完整参数表

产品参数
Product Parameters

型号	LT-USB 1080P	LT-USB 5MP
sensor规格	1/2.7inch -1/2.9inch	1/2.8inch
像素尺寸	Pixel Size 3um*3um	Pixel Size 2um*2um
帧率	30帧/秒	30帧/秒
硬件	高清200万像素	高清500万像素
供电方式	USB bus power	USB bus power
工作电流	约150-200mA	约110-190mA
动态范围	69dB	
信噪比	39dB	
工作电压	5V	
输出格式	MJPG(出厂默认格式：MJPG格式)支持YUY2(YUYV)	
对焦	手动对焦(可通过旋转镜头对准清晰度)	
镜头	根据自己的需求选择自己所需的焦距 2.8-12mm 5-50mm 5-100mm	
支持免驱协议	USB接口免驱动(USB2.0支持UVC通信协议)	
线材	1.5M、3M (默认发1.5M线，(需其他线长联系客服)	
成像距离	1CM至无限远	
图像处理	自动曝光/自动增益/自动白平衡	
图像控制	饱和度控制/锐度控制/亮度控制/对比度控制/伽码控制/白平衡	
工作湿度	避免在过于潮湿的环境使用，适合湿度范围(85%RH)以下内使用	
工作温度	避免在过于热或过冷环境使用，适合温度范围(-30°℃~+70℃)内使用	
快门类型	Electronic rolling shutter/Frame exposure	
支持系统	WindowsXP、Vista、win7、win8、win10 Linux Uduntu Android4.0(安卓)以上 MAC OS 树莓派	
适用环境	室内、室外	
适用设备	工业设备、媒体设备、网络视频会议、监控录像.POS机、 医疗设备、智能家居、展示台、查询机、自助设备、 一体机设备、网络电视盒、网络机顶盒、安卓设备、广告机、 工业工控电脑、ATM柜员机.等一系列设备	

蓝天科技

附录四　实验相关代码

附录 4.1 YOLO 模型加载与条纹目标检测

```
# src/vision/detector.py
    def load(self) -> bool:
        path = Path(self.model_path)
        if not path.exists():
            logger.error(f"模型文件不存在: {self.model_path}")
            return False
        try:
            if self.device.startswith("cuda") and not
torch.cuda.is_available():
                logger.warning("CUDA 不可用, YOLO 将回退到 CPU")
                self.device = "cpu"
            self._model = YOLO(str(path))
            logger.info(
                f"YOLO 模型已加载: {path.resolve()} "
                f"device={self.device} classes={self.class_names}"
            )
            return True
        except Exception as exc:
            self._model = None
            logger.exception(f"YOLO 模型加载失败: {exc}")
            return False

    def detect(
        self,
        frame: np.ndarray,
        roi: tuple[int, int, int, int] | None = None,
    ) -> dict:
        if self._model is None:
            logger.warning("YOLO 模型未加载")
            return self._empty_result()

        # ROI 裁剪
        target = frame
        roi_offset = (0, 0)
        if roi is not None:
            x, y, w, h = roi
            height, width = frame.shape[:2]
            x, y = max(0, int(x)), max(0, int(y))
            if x >= width or y >= height:
```

```
                logger.warning(f"ROI 起点超出图像范围: {roi}")
                return self._empty_result(frame)
            w = min(max(0, int(w)), width - x)
            h = min(max(0, int(h)), height - y)
            if w < 10 or h < 10:
                logger.warning(f"ROI 尺寸无效: {(x, y, w, h)}")
                return self._empty_result(frame)
            target = frame[y:y + h, x:x + w]
            roi_offset = (x, y)

        if target.size == 0:
            return self._empty_result(frame)

        try:
            results = self._model.predict(
                target, conf=min(max(self.confidence, 0.0), 1.0),
                iou=min(max(self.iou, 0.0), 1.0),
                imgsz=max(32, int(self.imgsz)), device=self.device,
                verbose=False,
            )
        except Exception as exc:
            logger.exception(f"YOLO 推理失败: {exc}")
            return self._empty_result(frame, error=str(exc))
        result = results[0]

        boxes_xyxy = result.boxes.xyxy.cpu().numpy() if result.boxes is not
None else np.array([])
        confs = result.boxes.conf.cpu().numpy() if result.boxes is not None
else np.array([])
        class_ids    =    result.boxes.cls.cpu().numpy().astype(int)    if
result.boxes is not None else np.array([])
        class_names = [result.names.get(cid, "unknown") for cid in class_ids]

        # 坐标映射回全帧
        if roi_offset != (0, 0) and len(boxes_xyxy) > 0:
            ox, oy = roi_offset
            boxes_xyxy[:, [0, 2]] += ox
            boxes_xyxy[:, [1, 3]] += oy

        # 在全帧上画框
        annotated = frame.copy()
        for i in range(len(boxes_xyxy)):
            x1, y1, x2, y2 = boxes_xyxy[i].astype(int)
            cv2.rectangle(annotated, (x1, y1), (x2, y2), (0, 255, 255), 2)
```

```
            label = f"{class_names[i]} {confs[i]:.2f}"
            cv2.putText(annotated, label, (x1, max(18, y1 - 6)),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 255, 255), 2)

        # 最高置信度框的中心
        center = None
        if len(boxes_xyxy) > 0:
            best_idx = int(np.argmax(confs))
            bx1, by1, bx2, by2 = boxes_xyxy[best_idx]
            center = ((bx1 + bx2) / 2, (by1 + by2) / 2)

        return {
            "boxes_xyxy": boxes_xyxy,
            "confs": confs,
            "class_ids": class_ids,
            "class_names": class_names,
            "annotated": annotated,
            "center": center,
        }
```

附录 4.2 中央条纹定位与连续帧跟踪

```
# src/vision/fringe_center.py
def find_center_in_region(
    bgr_roi: np.ndarray,
    expected_center_x: float | None = None,
    search_radius: float | None = None,
    search_bounds: tuple[float, float] | None = None,
) -> dict:
    """定位白光干涉竖条纹的相干包络中心。

    中心条纹可以是黑、白、蓝或其他颜色。算法不搜索某一种颜色，而是
    联合三个颜色通道的局部条纹能量、包络左右对称性和可选的 YOLO 位置
    先验。``search_bounds`` 用于指定 YOLO 零级框在 ROI 内的左右边界；框
    仅限定搜索区域，不假设框的几何中心就是物理中心条纹。所有坐标均相对
    于 ``bgr_roi``。
    """
    if not isinstance(bgr_roi, np.ndarray) or bgr_roi.size == 0:
        raise ValueError("中心检测输入为空")
    if bgr_roi.ndim not in (2, 3):
        raise ValueError(f"不支持的图像维度: {bgr_roi.ndim}")
    height, width = bgr_roi.shape[:2]
    if min(height, width) < 20:
        raise ValueError(f"中心检测区域过小: {bgr_roi.shape[:2]}")
```

```
    profiles, verticality = _profiles_for_vertical_stripes(bgr_roi)
    dynamic_range = float(max(np.percentile(p, 95) - np.percentile(p, 5) for
p in profiles))
    if dynamic_range < 0.03:
        raise ValueError("图像对比度过低，未检测到有效条纹")

    luma = 0.114 * profiles[0] + 0.587 * profiles[1] + 0.299 * profiles[2]
    period = _estimate_period(luma)
    background_sigma = max(period * 1.8, width / 18.0, 5.0)
    high_pass = np.vstack([
        channel - ndimage.gaussian_filter1d(channel, background_sigma)
        for channel in profiles
    ])

    # 相干区的多通道振荡能量高；中心颜色改变不会改变这一几何特征。
    energy = np.sum(high_pass ** 2, axis=0)
    gradient = np.sum(np.gradient(high_pass, axis=1) ** 2, axis=0)
    envelope_sigma = max(2.0, period * 0.85)
    energy = ndimage.gaussian_filter1d(energy, envelope_sigma)
    gradient = ndimage.gaussian_filter1d(gradient, envelope_sigma)
    coherence = 0.68 * _normalise(energy) + 0.32 * _normalise(gradient)

    margin = max(3, int(round(period / 2)))
    lo, hi = margin, width - margin
    expected = None
    if expected_center_x is not None and np.isfinite(expected_center_x):
        expected = float(np.clip(expected_center_x, 0, width - 1))

    bounded_by_zero_box = search_bounds is not None
    if search_bounds is not None:
        bound_left,    bound_right    =    sorted((float(search_bounds[0]),
float(search_bounds[1])))

# ……非核心界面、日志或重复校验代码省略……

        prior[lo:hi] = 0.5
    else:
        # 只作弱平局判据；允许物理中心偏离 YOLO 框中心多个条纹周期。
        prior_sigma = max(3.0, (hi - lo) * (0.50 if bounded_by_zero_box else
0.28))
        xx = np.arange(width, dtype=np.float64)
        prior = np.exp(-0.5 * ((xx - expected) / prior_sigma) ** 2)
```

```
    if bounded_by_zero_box:
        box_interior = np.zeros(width, dtype=np.float64)
        box_width = max(bound_right - bound_left, 1.0)
        relative_x = np.clip((np.arange(width) - bound_left) / box_width,
0.0, 1.0)
        # 只抑制紧贴框线的结构，不把候选强拉回几何中心。
        box_interior = np.sqrt(np.maximum(np.sin(np.pi * relative_x), 0.0))
        score = (
            0.25 * darkness
            + 0.30 * chroma
            + 0.16 * stripe_strength
            + 0.12 * edge_pair
            + 0.10 * symmetry
            + 0.03 * prior
            + 0.04 * box_interior
        )
    elif expected is None:
        score = 0.78 * coherence + 0.22 * symmetry
    else:
        # 兼容没有传零级框边界的旧调用：仍采用保守的亚周期精修。
        score = 0.25 * coherence + 0.10 * symmetry + 0.65 * prior

    best = lo + int(np.argmax(score[lo:hi]))
    # 三点抛物线作亚像素精修，避免整数像素来回跳。
    center = float(best)
    if 0 < best < width - 1:
        a, b, c = score[best - 1:best + 2]
        denom = a - 2 * b + c
        if abs(denom) > 1e-9:
            center += float(np.clip(0.5 * (a - c) / denom, -0.5, 0.5))

    peak = float(score[best])
    baseline = float(np.median(score[lo:hi]))
    prominence = np.clip((peak - baseline) / max(1.0 - baseline, 1e-6), 0.0,
1.0)
    contrast_quality = np.clip(dynamic_range / 0.18, 0.0, 1.0)
    orientation_quality = np.clip((verticality - 0.30) / 0.35, 0.0, 1.0)
    prior_quality = 0.85 if expected is not None else 0.65
    confidence = contrast_quality * (0.35 + 0.65 * prominence) * (
        0.45 + 0.55 * orientation_quality
    ) * prior_quality

    # 只有明显由横向结构主导时才拒绝；轻微倾斜的竖条纹仍视为有效。
    orientation = "horizontal" if verticality < 0.30 else "vertical"
```

```
    return {
        "center_main": center,
        "center_x": center,
        "center_y": float(height / 2),
        "confidence": float(np.clip(confidence, 0.0, 1.0)),
        "orientation": orientation,
        "verticality": verticality,
        "period": float(period),
        "roi_width": width,
        "roi_height": height,
        "methods": {
            "coherence": float(np.argmax(coherence[lo:hi]) + lo),
            "symmetry": float(np.argmax(symmetry[lo:hi]) + lo),
            "stripe_candidate": float(best),
            "darkness": float(darkness[best]),
            "chroma": float(chroma[best]),
            "edge_pair": float(edge_pair[best]),
            "prior": float(expected) if expected is not None else
float("nan"),
        },
    }



# ……非核心界面、日志或重复校验代码省略……

    def update(self, center: float | None, confidence: float = 0.0) -> dict:
        if center is None or not np.isfinite(center):
            self._misses += 1
            if self._center is None or self._misses > self.hold_frames:
                self.reset()
                return {"center": None, "confidence": 0.0, "held": False,
"accepted": False}
            return {
                "center": self._center,
                "confidence": self._confidence * (0.78 ** self._misses),
                "held": True,
                "accepted": False,
            }

        value = float(center)
        conf = float(np.clip(confidence, 0.0, 1.0))
        if self._center is not None and abs(value - self._center) >
self.max_jump_px and conf < 0.78:
```

```
            return self.update(None, 0.0)

        self._misses = 0
        self._history.append(value)
        median = float(np.median(self._history))
        self._center = median if self._center is None else 0.55 *
self._center + 0.45 * median
        self._confidence = conf
        return {"center": self._center, "confidence": conf, "held": False,
"accepted": True}
```

附录 4.3 运动过程条纹增强

```
# src/vision/motion_enhancement.py
    def apply(
        self,
        frame: np.ndarray,
        *,
        clarity_score: float | None = None,
        clarity_baseline: float | None = None,
        blur_ratio: float = 0.55,
    ) -> np.ndarray:
        """返回同尺寸 BGR 增强帧；输入帧不会被原地修改。"""
        if not self.enabled or frame is None or frame.size == 0:
            self.last_strength = 0.0
            self.last_stripe_strength = 0.0
            self.last_color_gain = 1.0
            return frame

        blur_amount = self._blur_amount(
            clarity_score, clarity_baseline, blur_ratio)
        strength = self.base_strength + (
            self.max_strength - self.base_strength) * blur_amount
        stripe_strength = self.stripe_strength + (
            self.max_stripe_strength - self.stripe_strength) * blur_amount
        color_gain = self.color_gain + (
            self.max_color_gain - self.color_gain) * blur_amount

        # 先沿竖直条纹方向平均噪声，再减去较宽的水平背景。
        # 这样能从不均匀照明中提取低对比度、近似周期性的竖条纹。
        vertical_smooth = cv2.GaussianBlur(
            frame, (1, self.vertical_size), sigmaX=0, sigmaY=0)
        horizontal_background = cv2.GaussianBlur(
            vertical_smooth, (self.background_size, 1), sigmaX=0)
```

```
        stripe_layer = cv2.addWeighted(
            vertical_smooth, 1.0 + stripe_strength,
            horizontal_background, -stripe_strength, 0)
        restored = cv2.addWeighted(
            frame, self.original_mix,
            stripe_layer, 1.0 - self.original_mix, 0)

        # 白光干涉条纹主要为竖向结构，旋转时拖影主要沿水平方向。
        # 仅恢复水平方向高频，比二维锐化更少放大上下边缘和传感器噪声。
        horizontal_blur = cv2.GaussianBlur(
            restored, (self.kernel_size, 1), sigmaX=0)
        restored = cv2.addWeighted(
            restored, 1.0 + strength, horizontal_blur, -strength, 0)

        # 放大相对于灰度分量的颜色差异，使白光干涉的弱彩色条纹更明显。
        gray = cv2.cvtColor(restored, cv2.COLOR_BGR2GRAY)
        gray_bgr = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
        restored = cv2.addWeighted(
            restored, color_gain, gray_bgr, 1.0 - color_gain, 0)
        if abs(self.contrast_gain - 1.0) > 0.001:
            restored            =            cv2.convertScaleAbs(restored,
alpha=self.contrast_gain)

        self.last_strength = strength
        self.last_stripe_strength = stripe_strength
        self.last_color_gain = color_gain
        return restored

    @staticmethod
```

附录 4.4 数显微分表定位、OCR 识别与稳定读数

```
# src/vision/micrometer_ocr.py
def locate_lcd(
    frame: np.ndarray,
    *,
    auto_roi: bool = True,
    manual_roi: Iterable[float] = (0.0, 0.0, 1.0, 1.0),
) -> tuple[np.ndarray, tuple[int, int, int, int]]:
    """定位长条形 LCD；未找到时回退到配置的手动 ROI。"""
    if frame is None or frame.size == 0:
        raise ValueError("摄像头画面为空")
    fallback = _manual_roi(frame, manual_roi)
    if not auto_roi:
```

```
        x1, y1, x2, y2 = fallback
        return frame[y1:y2, x1:x2].copy(), fallback

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY) if frame.ndim == 3 else
frame
    clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8)).apply(gray)
    edges = cv2.Canny(cv2.GaussianBlur(clahe, (5, 5), 0), 35, 120)
    edges = cv2.morphologyEx(
        edges,  cv2.MORPH_CLOSE,  cv2.getStructuringElement(cv2.MORPH_RECT,
(9, 5)))
    contours,      _      =      cv2.findContours(edges,      cv2.RETR_LIST,
cv2.CHAIN_APPROX_SIMPLE)
    height, width = gray.shape[:2]
    frame_area = float(height * width)
    best: tuple[float, tuple[int, int, int, int]] | None = None
    for contour in contours:
        x, y, rw, rh = cv2.boundingRect(contour)
        if rh < 30 or rw < 120:
            continue
        aspect = rw / max(1.0, float(rh))
        area_ratio = (rw * rh) / frame_area
        if not 2.0 <= aspect <= 6.8 or not 0.015 <= area_ratio <= 0.75:
            continue
        contour_area = cv2.contourArea(contour)
        rectangularity = contour_area / max(1.0, float(rw * rh))
        if rectangularity < 0.15:
            continue
        cx, cy = x + rw / 2.0, y + rh / 2.0
        center_distance = abs(cx / width - 0.5) + abs(cy / height - 0.5)
        score = area_ratio * 5.0 + rectangularity - center_distance * 0.25
        if best is None or score > best[0]:
            margin_x = int(rw * 0.025)
            margin_y = int(rh * 0.06)
            box = (
                max(0, x + margin_x),
                max(0, y + margin_y),
                min(width, x + rw - margin_x),
                min(height, y + rh - margin_y),
            )
            best = score, box

    box = best[1] if best is not None else fallback
    x1, y1, x2, y2 = box
    crop = frame[y1:y2, x1:x2]
```

```
    if crop.size == 0:
        x1, y1, x2, y2 = fallback
        crop = frame[y1:y2, x1:x2]
        box = fallback
    return crop.copy(), box



# ……非核心界面、日志或重复校验代码省略……

    def update(self, value: float | None) -> tuple[float | None, bool, int]:
        self.last_rejected = False
        self.last_reason = ""
        self.last_candidate = None if value is None else round(
            float(value), self.decimal_places)
        if value is None:
            return self.stable_value, False, 0
        rounded = round(float(value), self.decimal_places)

        if self.stable_value is not None:
            jump = abs(rounded - self.stable_value) > self.max_step
            # 靠近零点时，两个完全正常的三位小数值也可能恰好相差约
            # 10 倍（例如 -0.021 → -0.002）。只有比例异常同时伴随
            # 足够大的绝对跳变时，才判断为小数点/位数误识别。
            if jump and self._is_scale_error(rounded):
                self.last_rejected = True
                self.last_reason = "疑似 ×10/÷10 小数点或位数误识别"
                self._values.clear()
                self._jump_candidate = None
                self._jump_count = 0
                return self.stable_value, False, 0

            if jump:
                if rounded == self._jump_candidate:
                    self._jump_count += 1
                else:
                    self._jump_candidate = rounded
                    self._jump_count = 1
                if self._jump_count < self.jump_required:
                    self.last_rejected = True
                    self.last_reason = (
                        f"      读      数      突      变      超      过
{self.max_step:.{self.decimal_places}f} mm")
                    self._values.clear()
```

```
                    return self.stable_value, False, self._jump_count
                # 极端情况下真实位置可能在掉帧期间发生变化；只有很多帧
                # 持续一致才允许建立新的基准，避免永久锁死旧值。
                self.stable_value = rounded
                self._values.clear()
                self._jump_candidate = None
                self._jump_count = 0
                return self.stable_value, True, self.jump_required

            self._jump_candidate = None
            self._jump_count = 0

        self._values.append(rounded)
        recent = list(self._values)[-self.required:]
        count = 0
        for previous in reversed(self._values):
            if previous != rounded:
                break
            count += 1
        stable = len(recent) >= self.required and all(
            previous == rounded for previous in recent)
        if stable:
            self.stable_value = rounded
        return self.stable_value, stable, count



# ……非核心界面、日志或重复校验代码省略……

    def recognize(
        self,
        frame: np.ndarray,
        *,
        auto_roi: bool = True,
        manual_roi: Iterable[float] = (0.0, 0.0, 1.0, 1.0),
    ) -> MicrometerOCRResult:
        self.load()
        crop, box = locate_lcd(
            frame, auto_roi=auto_roi, manual_roi=manual_roi)
        variants = [crop]
        gray = cv2.cvtColor(crop, cv2.COLOR_BGR2GRAY) if crop.ndim == 3 else
crop
        enhanced    =    cv2.createCLAHE(clipLimit=2.5,    tileGridSize=(8,
4)).apply(gray)
```

```
        variants.append(cv2.cvtColor(enhanced, cv2.COLOR_GRAY2BGR))

        best_text, best_score = "", 0.0
        best_normalized: str | None = None
        best_rank = -1.0
        best_hint = ""
        for image in variants:
            output = self._engine(
                image, use_det=False, use_cls=False, use_rec=True)
            texts = tuple(getattr(output, "txts", ()) or ())
            scores = tuple(getattr(output, "scores", ()) or ())
            if not texts or not scores:
                continue
            raw_text = str(texts[0])
            raw_score = float(scores[0])
            normalized = normalize_meter_text(raw_text, self.decimal_places)
            normalized_raw = raw_text.strip().replace(",", ".").replace(" ",
"")
            explicit_decimal = "." in normalized_raw
            fraction_digits = (
                len(normalized.rsplit(".", 1)[1])
                if normalized is not None and "." in normalized else 0
            )
            # 三位小数只参与候选排序，不作为硬性通过条件。明确识别到的
            # 小数点优先；小数点缺失时允许按末三位补回，但稍降排序权重。
            hint = ""
            format_adjustment = 0.0
            if normalized is None:
                format_adjustment = -0.1
            elif explicit_decimal:
                if fraction_digits == self.decimal_places:
                    format_adjustment = 0.025
                else:
                    format_adjustment = -0.015
                    hint = f"画面结果不是{self.decimal_places}位小数"
            elif normalized is not None and self.decimal_places > 0:
                format_adjustment = -0.01
                hint = f"小数点按末{self.decimal_places}位辅助补全"
            rank = raw_score + format_adjustment
            if rank > best_rank:
                best_rank = rank
                best_text = raw_text
                best_score = raw_score
                best_normalized = normalized
```

```
                best_hint = hint

        normalized = best_normalized
        value = None
        if normalized is not None and best_score >= self.min_score:
            try:
                value = float(normalized)
            except ValueError:
                value = None
        stable_value, stable, repeat_count = self.stabilizer.update(value)
        held = stable_value is not None and not stable
        if value is None:
            message = (
```

附录 4.5 已知方向搜索与条纹闭环寻中

```
# src/control/center_control.py
    def start(self, now: float) -> CenterControlDecision:
        self.enabled = True
        self.started_at = float(now)
        self.direction = "stopped"
        self.gear = None
        self.stable_frames = 0
        self.missing_frames = 0
        self.center_seen = False
        self.center_candidate_frames = 0
        self.motion_updates = 0
        self.forward_x_sign = 0
        self.direction_learning_score = 0
        self.learning_reference_x = None
        self.learning_direction = "stopped"
        self.guide_seen = False
        self.guide_missing_frames = 0
        self.featureless_frames = 0
        self.blur_recovery_active = False
        self.clear_visual_frames = 0
        self.guide_last_distance = None
        self.guide_history.clear()
        self.search_planner_configured = False
        self.search_last_update_at = float(now)
        self.guide_initial_direction_selected = False
        self.guide_focus_direction = ""
        self.guide_focus_frames = 0
        self.guide_focus_last_level = -1
```

```
        return CenterControlDecision(
            state="searching", message="正在搜索中心条纹")

    def stop(self, reason: str = "用户停止") -> CenterControlDecision:
        was_moving = self.direction != "stopped"
        was_enabled = self.enabled
        self.enabled = False
        self.direction = "stopped"
        self.gear = None
        self.stable_frames = 0
        commands = (("stop", None),) if was_enabled or was_moving else ()
        return CenterControlDecision(
            commands=commands,
            state="stopped",
            message=reason,
            stopped_reason=reason if was_enabled else "",
            **self._range_fields("stopped"),
        )


# ……非核心界面、日志或重复校验代码省略……

    def _single_direction_decision(
        self, direction: str, gear: int, *, message: str,
        phase: str, error: float | None = None,
    ) -> CenterControlDecision:
        """在中心条纹尚未出现时沿已知方向连续搜索。"""
        commands = self._motion_commands(direction, gear)
        return CenterControlDecision(
            commands=commands,
            state="single_direction_search",
            message=f"{message}（找到中心条纹前不往返）",
            error_px=error,
            direction=direction,
            gear=gear,
            direction_mapping=f"固定{self._direction_text(direction)}",
            **self._range_fields(phase),
        )


# ……非核心界面、日志或重复校验代码省略……

    def _dynamic_search_gear(self, base_gear: int, min_gear: int,
                             acceleration_step: int) -> int:
```

```
        """每完成一轮双侧搜索便提高速度，档位数字越小越快。"""
        fastest_allowed = min(self._gear(base_gear), self._gear(min_gear))
        return max(
            fastest_allowed,
            self._gear(base_gear)
            -       self.search_planner.expansion_level       *       max(0,
acceleration_step),
        )

    def _learn_direction(self, center_x: float, minimum_delta: float) ->
None:
        """根据已知运动方向和条纹位移学习正转对应的画面方向。"""
        if self.direction not in ("forward", "reverse"):
            self.learning_reference_x = center_x
            self.learning_direction = "stopped"
            return
        if (self.learning_reference_x is None
                or self.learning_direction != self.direction):
            self.learning_reference_x = center_x
            self.learning_direction = self.direction
            return
        delta = center_x - self.learning_reference_x
        if abs(delta) < minimum_delta:
            return
        observed_sign = 1 if delta > 0 else -1
        forward_sign = observed_sign if self.direction == "forward" else -
observed_sign
        self.direction_learning_score += forward_sign
        self.direction_learning_score        =        max(-3,        min(3,
self.direction_learning_score))
        self.forward_x_sign = 1 if self.direction_learning_score > 0 else -
1
        self.learning_reference_x = center_x

    def _advance_search_position(self, now: float) -> None:
        """按实际档位角速度积分虚拟位置，供扩展扫描规划使用。"""
        dt = max(0.0, float(now) - self.search_last_update_at)
        self.search_last_update_at = float(now)
        if dt <= 0 or self.direction not in ("forward", "reverse"):
            return
        gear = MOTOR_GEAR_TABLE.get(int(self.gear or 0))
        if not gear:
            return
        turns = dt / float(gear["turn_seconds"])
```

```
        signed_turns = turns if self.direction == "forward" else -turns
        self.search_planner.advance(signed_turns)


# ……非核心界面、日志或重复校验代码省略……

    def _motion_commands(
        self, direction: str, gear: int | None,
    ) -> tuple[tuple[str, int | None], ...]:
        commands: list[tuple[str, int | None]] = []
        if direction == "stopped":
            if self.direction != "stopped":
                commands.append(("stop", None))
            self.direction = "stopped"
            self.gear = None
            self.motion_updates = 0
            return tuple(commands)

        if self.direction not in ("stopped", direction):
            commands.append(("stop", None))
            self.direction = "stopped"
        if gear is not None and gear != self.gear:
            commands.append(("set_speed", gear))
            self.gear = gear
        if self.direction != direction:
            commands.append((f"start_{direction}", None))
            self.direction = direction
            self.motion_updates = 0
        else:
            self.motion_updates += 1
            if self.motion_updates >= self.command_refresh_frames:
                commands.append((f"start_{direction}", None))
                self.motion_updates = 0
        return tuple(commands)

    @staticmethod
```

附录 4.6 电机串口连接与运动控制

```
# src/hardware/motor.py
    def connect(self) -> bool:
        selected_port = self.detect_port(self.port)
        if selected_port is None:
            logger.error("未检测到可确定的电机串口")
```

```
            return False
        self.port = selected_port
        try:
            self._ser     =     serial.Serial(self.port,     self.baudrate,
timeout=self.timeout)
            self._connected = True
            logger.info("电机已连接: %s", self.port)
            return True
        except serial.SerialException as exc:
            logger.error("电机连接失败 %s: %s", self.port, exc)
            return False

    def close(self) -> None:
        if self._ser and self._ser.is_open:
            self._ser.close()
        self._connected = False
        logger.info("电机已断开")

    @property
    def is_connected(self) -> bool:
        return   self._connected   and   self._ser   is   not   None   and
self._ser.is_open

    def send_cmd(self, cmd: str, read_response: bool = False) -> str | bool
| None:
        """发送单个 ASCII 字符，可选读取状态响应。

        部分控制器会先回显查询字符，再在下一行返回 JSON，因此查询时最多
        读取四行，并在获得可识别的状态内容后立即返回。
        """
        if not self.is_connected:
            logger.warning("电机未连接")
            return None
        if not isinstance(cmd, str) or len(cmd) != 1 or ord(cmd) > 127:
            raise ValueError("电机命令必须是单个 ASCII 字符")
        try:
            with self._io_lock:
                if       read_response       and       hasattr(self._ser,
"reset_input_buffer"):
                    self._ser.reset_input_buffer()
                self._ser.write(cmd.encode("ascii"))
                if read_response:
                    lines = []
                    for _ in range(4):
```

```
                        raw = self._ser.readline()
                        if not raw:
                            break
                        line = raw.decode("utf-8", errors="replace").strip()
                        if not line:
                            continue
                        lines.append(line)
                        if self._looks_like_status(line):
                            break
                    return "\n".join(lines)
            return True
        except serial.SerialException as exc:
            self._connected = False
            logger.error("电机命令失败: %s", exc)
            return None

    def start_forward(self) -> bool:
        return self.send_cmd(MotorCommand.FORWARD.value) is True

    def start_reverse(self) -> bool:
        return self.send_cmd(MotorCommand.REVERSE.value) is True

    def start(self) -> bool:
        """自动实验兼容入口，默认按正转方向启动。"""
        return self.start_forward()

    def stop(self) -> bool:
        return self.send_cmd(MotorCommand.STOP.value) is True

    def toggle_direction(self) -> bool:
        return self.send_cmd(MotorCommand.TOGGLE_DIRECTION.value) is True

    def speed_up(self) -> bool:
        return self.send_cmd(MotorCommand.SPEED_UP.value) is True

    def speed_down(self) -> bool:
        return self.send_cmd(MotorCommand.SPEED_DOWN.value) is True

    def query_status(self) -> dict:
        response          =          self.send_cmd(MotorCommand.QUERY.value,
read_response=True)
        if not isinstance(response, str):
            return self._empty_status()
        return self._parse_status(response)
```

```
    def set_speed(self, target_speed: int, max_attempts: int = 12) -> bool:
```

附录 4.7 DeepSeek 实验助手问答

```
# src/agent/service.py
    def ask(self, question: str, include_status: bool = True,
            context_override: dict | None = None,
            cancel_event: threading.Event | None = None) -> AgentResponse:
        question = question.strip()
        if not question:
            return AgentResponse("请输入问题。", (), False)
        chunks = self.knowledge.search(question, self.top_k)
        if include_status and context_override is not None:
            context = context_override
        else:
            context = self.context_provider() if include_status and
self.context_provider else {}
        if not chunks and not self.provider.available and not context:
            return AgentResponse(
                "本地知识库中没有找到足够相关的资料。请换一种表述，或补充实
验现象和当前步骤。",
                (), False, "无检索结果")
        if not self.provider.available:
            answer = self._offline_answer(chunks, context)
            self._remember(question, answer)
            return AgentResponse(answer, tuple(chunks), False,
                                 "未配置 API Key，已使用本地检索回答")
        references = "\n\n".join(
            f"[来源{i}] {chunk.title}\n{chunk.text}"
            for i, chunk in enumerate(chunks, 1))
        status_json = json.dumps(context, ensure_ascii=False,
separators=(",", ":"))
        if len(status_json) > self.context_max_chars:
            status_json = status_json[:self.context_max_chars] + "…[状态上
下文已截断]"
        status_text = ("\n 当前实验状态：" + status_json) if context else ""
        if not references:
            references = "本地知识库未命中。只能回答一般性问题；涉及具体实
验事实时应要求用户补充资料。"
        messages = [{"role": "system", "content": SYSTEM_PROMPT}]
        with self._history_lock:
            history = list(self._history)
        for old_question, old_answer in history:
```

```
            messages.extend((
                {"role":                  "user",                  "content":
old_question[:self.history_question_chars]},
                {"role":               "assistant",               "content":
old_answer[:self.history_answer_chars]},
            ))
        messages.append({
            "role": "user",
            "content": f"问题：{question}{status_text}\n\n参考资料：
\n{references}",
        })
        try:
            report_request = any(keyword in question.lower() for keyword in
(
                "实验报告", "生成报告", "报告模板", "report"))
            output_budget   =   max(self.provider.max_tokens,   3000)   if
report_request else None
            answer = self.provider.chat(
                messages,                          cancel_event=cancel_event,
max_tokens=output_budget)
            self._remember(question, answer)
            return AgentResponse(answer, tuple(chunks), True)
        except ProviderCancelled:
            raise
        except ProviderError as exc:
            fallback = self._offline_answer(chunks, context) if chunks else
(
                "在线模型调用失败，且本地知识库没有命中相关资料。请检查连接
状态或补充资料。")
            return AgentResponse(fallback, tuple(chunks), False, str(exc))

# src/agent/provider.py
    def chat(self, messages: list[dict[str, str]],
             cancel_event: threading.Event | None = None,
             max_tokens: int | None = None) -> str:
        if not self.available:
            raise ProviderError("未配置 API Key")
        if cancel_event is not None and cancel_event.is_set():
            raise ProviderCancelled("请求已取消")
        payload = json.dumps({
            "model": self.model,
            "messages": messages,
            "temperature": 0.2,
            "max_tokens": int(max_tokens or self.max_tokens),
```

```
            "stream": True,
        }, ensure_ascii=False).encode("utf-8")
        request = urllib.request.Request(
            f"{self.api_base}/chat/completions",
            data=payload,
            headers={
                "Authorization": f"Bearer {self.api_key}",
                "Content-Type": "application/json",
            },
            method="POST",
        )
        try:
            with urllib.request.urlopen(request, timeout=self.timeout) as
response:
                parts = []
                for raw_line in response:
                    if cancel_event is not None and cancel_event.is_set():
                        raise ProviderCancelled("请求已取消")
                    line = raw_line.decode("utf-8",
errors="replace").strip()
                    if not line.startswith("data:"):
                        continue
                    data_text = line[5:].strip()
                    if data_text == "[DONE]":
                        break
                    data = json.loads(data_text)
                    delta = data["choices"][0].get("delta", {})
                    content = delta.get("content", "")
                    if content:
                        parts.append(content)
            answer = "".join(parts).strip()
            if not answer:
                raise ProviderError("模型返回了空响应")
            return answer
        except urllib.error.HTTPError as exc:
            try:
                detail = exc.read().decode("utf-8", errors="replace")[:300]
            except Exception:
                detail = ""
            raise ProviderError(f"模型调用失败: HTTP {exc.code} {detail}")
from exc
```

附录 4.8 条纹仿真

```
% ========================================================================
%  三种光源的迈克尔逊等厚干涉直条纹仿真
%
%  物理模型:
%    - 楔形空气膜等厚干涉: 光程差 Δ = 2d = 2αx
%    - 半波损失: I(λ,d) = S(λ) × sin²(2πd/λ) → 零级为暗纹
%    - 连续光谱求和: 实测光谱 S(λ), 380-780 nm, ~1 nm 步长
%    - 波长→sRGB 颜色映射 (Bruton 算法, γ=0.80)
%
%  数据来源: LET1 卤素灯 / LET2 白炽灯 / LET3 LED
%  输出: 三种光源的干涉直条纹图样 (独立 PNG 文件)
%
%  作者: 基于 乔亮 (2015) 迈克尔逊干涉实验的仿真研究
% ========================================================================

clear; clc; close all;

%% ===== 1. 读取实测光谱数据 =====
base = 'd:/光谱分析/光谱/';

d1 = load([base 'LET1.txt']);   % 卤素灯
d2 = load([base 'LET2.txt']);   % 白炽灯
d3 = load([base 'LET3.txt']);   % LED

wl_raw = d1(:, 1);              % 波长 (nm)
I1_raw = d1(:, 2);              % 卤素灯相对强度
I2_raw = d2(:, 2);              % 白炽灯相对强度
I3_raw = d3(:, 2);              % LED 相对强度

% 截取可见光波段 380-780 nm
vis = (wl_raw >= 380) & (wl_raw <= 780);
wl  = wl_raw(vis);
I1  = I1_raw(vis);
I2  = I2_raw(vis);
I3  = I3_raw(vis);
N_wl = length(wl);

fprintf('光谱数据加载: %d 个波长点 (%.0f - %.0f nm)\n', N_wl, wl(1),
wl(end));
fprintf('  卤素灯:  峰值 %.0f nm, 强度 [%.0f, %.0f]\n', wl(I1==max(I1)),
min(I1), max(I1));
fprintf('  白炽灯:  峰值 %.0f nm, 强度 [%.0f, %.0f]\n', wl(I2==max(I2)),
min(I2), max(I2));
fprintf('  LED:     峰值 %.0f nm, 强度 [%.0f, %.0f]\n', wl(I3==max(I3)),
```

```
min(I3), max(I3));

%% ===== 2. 干涉参数 =====
Nx = 2000;                          % 水平像素数
x_max = 2.5e-3;                     % 视场范围 ±2.5 mm
x = linspace(-x_max, x_max, Nx);

alpha  = 2e-6 / x_max;              % 楔角 (~8×10⁻⁴ rad)
d_gap  = alpha * x;                 % 空气膜厚度 (m), d = 0 在中心

fprintf('\n干涉参数: 视场 ±%.1f mm, 空气膜厚度 ±%.1f μm\n', ...
    x_max*1e3, max(abs(d_gap))*1e6);

%% ===== 3. 干涉计算 (全光谱连续求和 + 半波损失) =====
sources = {I1, I2, I3};
names   = {'LET1 Halogen (~3000 K)', 'LET2 Incandescent (~2700 K)', ...
           'LET3 LED (GaN 452 nm + phosphor)'};
tags    = {'halogen', 'incandescent', 'led'};

results = cell(1, 3);

fprintf('\n===== 连续光谱干涉计算 =====\n');

for src = 1:3
    I_spec = sources{src};
    Irgb   = zeros(Nx, 3);

    fprintf('  %s ...', names{src});

    for i = 1:N_wl
        lambda = wl(i) * 1e-9;          % 波长 (m)
        wgt    = I_spec(i);              % 实测光谱权重

        % ---- 核心公式 ----
        % 迈克尔逊等厚干涉 + 半波损失:
        %   I ∝ sin²(2πd/λ)
        %   d = 0 时 I = 0 → 零级暗纹
        I_intf = sin(2 * pi * d_gap / lambda).^2;

        [r, g, b] = wl2rgb(wl(i));

        Irgb(:, 1) = Irgb(:, 1) + wgt * I_intf' * r;
        Irgb(:, 2) = Irgb(:, 2) + wgt * I_intf' * g;
        Irgb(:, 3) = Irgb(:, 3) + wgt * I_intf' * b;
```

```
    end

    % ---- 后处理 ----
    Irgb = Irgb / max(Irgb(:));

    if src == 3
        % LED: 白底高饱和风格
        %  ① 逐通道直方图拉伸 (1% - 99% → [0, 1])
        for ch = 1:3
            ch_data = Irgb(:, ch);
            lo = prctile(ch_data, 1);
            hi = prctile(ch_data, 99);
            if hi > lo
                Irgb(:, ch) = (ch_data - lo) / (hi - lo);
            end
        end
        Irgb = max(0, min(1, Irgb));

        %  ② 色彩增强 (拉大离灰度轴距离)
        gray = mean(Irgb, 2);
        Irgb = gray + 1.8 * (Irgb - gray);
        Irgb = max(0, min(1, Irgb));

        %  ③ 最终归一化
        Irgb = Irgb / max(Irgb(:));
    else
        % 热辐射源: 温和伽马校正
        Irgb = Irgb.^(1 / 1.4);
    end

    results{src} = Irgb;
    fprintf('  done\n');
end

%% ===== 4. 生成并保存干涉图样 =====
fprintf('\n===== 生成干涉条纹图样 =====\n');

% --- 4a. 卤素灯 + 白炽灯 (深色背景) ---
for src = [1, 2]
    Irgb_line = results{src};

    % 竖直扩展为条带
    strip_h = 500;
    I_strip = zeros(strip_h, Nx, 3);
```

```
    for ch = 1:3
        I_strip(:, :, ch) = repmat(Irgb_line(:, ch)', strip_h, 1);
    end

    % 布局
    bg_dark = [0.08 0.08 0.08];
    mt = 60;  mb = 80;  ms = 60;
    H = strip_h + mt + mb;
    W = Nx + 2 * ms;

    canvas = zeros(H, W, 3);
    for ch = 1:3, canvas(:, :, ch) = bg_dark(ch); end

    y1 = mt + 1;  y2 = mt + strip_h;
    x1 = ms + 1;  x2 = ms + Nx;
    canvas(y1:y2, x1:x2, :) = I_strip;

    img = uint8(canvas * 255);

    % 光源名称
    img = insertText(img, [ms+10, 10], names{src}, ...
        'Font', 'Microsoft YaHei', 'FontSize', 22, ...
        'TextColor', 'white', 'BoxColor', bg_dark, 'BoxOpacity', 0);

    % 物理说明
    img = insertText(img, [ms+10, H-mb+20], ...
        'd = 0 (zero-order dark fringe, half-wave loss)  |
sin^2(2πd/λ)', ...
        'FontSize', 14, 'TextColor', [200 200 200], ...
        'BoxColor', bg_dark, 'BoxOpacity', 0);

    % 刻度
    ty = H - mb + 5;
    cx = ms + round(Nx/2);
    img = insertShape(img, 'Line', [cx, ty, cx, ty+8], ...
        'Color', 'white', 'LineWidth', 2);
    lx = ms + round(Nx*(x_max-2e-3)/(2*x_max));
    rx = ms + round(Nx*(x_max+2e-3)/(2*x_max));
    img = insertShape(img, 'Line', [lx, ty, lx, ty+8], ...
        'Color', 'white', 'LineWidth', 1);
    img = insertShape(img, 'Line', [rx, ty, rx, ty+8], ...
        'Color', 'white', 'LineWidth', 1);

    fname = sprintf('C:\\Users\\landequming17\\michelson_%s.png',
```

```
tags{src});
    imwrite(img, fname);
    fprintf('  %s  (%d x %d)\n', fname, W, H);
end

% --- 4b. LED (白色背景, 高饱和) ---
Irgb_line = results{3};
led_h = 700;
I_strip = zeros(led_h, Nx, 3);
for ch = 1:3
    I_strip(:, :, ch) = repmat(Irgb_line(:, ch)', led_h, 1);
end

bg_white = [0.98 0.98 0.98];
mt = 30;  mb = 50;  ms = 40;
H = led_h + mt + mb;
W = Nx + 2 * ms;

canvas = zeros(H, W, 3);
for ch = 1:3, canvas(:, :, ch) = bg_white(ch); end

y1 = mt + 1;  y2 = mt + led_h;
x1 = ms + 1;  x2 = ms + Nx;
canvas(y1:y2, x1:x2, :) = I_strip;

img = uint8(canvas * 255);

% 标签
img = insertText(img, [ms+5, 6], 'LED  (GaN 452 nm + phosphor)', ...
    'Font', 'Microsoft YaHei', 'FontSize', 16, ...
    'TextColor', [60 60 60], 'BoxColor', bg_white, 'BoxOpacity', 0);

% 中心标记细线
cx = ms + round(Nx/2);
img = insertShape(img, 'Line', [cx, y1, cx, y2], ...
    'Color', [140 140 140], 'LineWidth', 1);

fname = 'C:\Users\landequming17\michelson_led.png';
imwrite(img, fname);
fprintf('  %s  (%d x %d)\n', fname, W, H);

%% ===== 5. 完成 =====
fprintf('\n========================================\n');
fprintf('  输出文件\n');
```

```
fprintf('========================================\n');
fprintf('  michelson_halogen.png       卤素灯干涉直条纹 (深底)\n');
fprintf('  michelson_incandescent.png 白炽灯干涉直条纹 (深底)\n');
fprintf('  michelson_led.png           LED 干涉直条纹    (白底)\n');
fprintf('\n 物理要点:\n');
fprintf('  - 半波损失 sin²(2πd/λ) → 零级暗纹\n');
fprintf('  - 连续光谱求和自然呈现相干性差异\n');
fprintf('  - LED GaN 蓝峰(~25 nm FWHM) → 长相干 → 鲜明彩色条纹\n');
fprintf('  - 热辐射源(宽带 ~300 nm) → 短相干 → 条纹集中于近场\n');

%% ======================================================================
function [R, G, B] = wl2rgb(lambda_nm)
% 可见光波长 (nm) → sRGB 颜色
% 参考: Dan Bruton's color science algorithm

    gamma = 0.80;

    if     lambda_nm >= 380 && lambda_nm < 440
        R = -(lambda_nm - 440) / 60;  G = 0;  B = 1;
    elseif lambda_nm >= 440 && lambda_nm < 490
        R = 0;  G = (lambda_nm - 440) / 50;  B = 1;
    elseif lambda_nm >= 490 && lambda_nm < 510
        R = 0;  G = 1;  B = -(lambda_nm - 510) / 20;
    elseif lambda_nm >= 510 && lambda_nm < 580
        R = (lambda_nm - 510) / 70;  G = 1;  B = 0;
    elseif lambda_nm >= 580 && lambda_nm < 645
        R = 1;  G = -(lambda_nm - 645) / 65;  B = 0;
    elseif lambda_nm >= 645 && lambda_nm <= 780
        R = 1;  G = 0;  B = 0;
    else
        R = 0;  G = 0;  B = 0;
    end

    % 人眼灵敏度因子
    if     lambda_nm >= 380 && lambda_nm < 420
        s = 0.3 + 0.7 * (lambda_nm - 380) / 40;
    elseif lambda_nm >= 420 && lambda_nm <= 700
        s = 1.0;
    elseif lambda_nm >  700 && lambda_nm <= 780
        s = 0.3 + 0.7 * (780 - lambda_nm) / 80;
    else
        s = 0;
    end
```

```
    R = safe_pow(R * s, gamma);
    G = safe_pow(G * s, gamma);
    B = safe_pow(B * s, gamma);
end

function y = safe_pow(x, g)
    if x > 0, y = x^g; else, y = 0; end
end
```